

# IA GENERATIVA

Ignacio Gómez  
2025

# IA Generativa

La **inteligencia artificial generativa (IA generativa)** es un tipo de inteligencia artificial capaz de crear contenido nuevo y original, como textos, imágenes, música, videos, código y otros formatos, basándose en patrones aprendidos a partir de grandes cantidades de datos previamente analizados.

## ¿Cómo funciona?

La IA generativa aprende a identificar patrones a partir de grandes volúmenes de datos (textos, imágenes o sonidos). Tras su entrenamiento, estos modelos pueden generar contenidos inéditos y únicos mediante la combinación y síntesis creativa de estos patrones aprendidos.

## Ejemplos comunes:

- **Texto:** ChatGPT, Gemini o DeepSeek generan textos o resúmenes, poemas, guiones, etc.
- **Imágenes:** DALL-E, Stable Diffusion o Midjourney generan imágenes a partir de descripciones textuales.
- **Audio:** Modelos que generan música original, voces sintéticas, o sonidos naturales.
- **Vídeo:** Creación automática de videos o animaciones realistas a partir de texto.

# IA Generativa

Los principales modelos de **Machine Learning (ML)** que se utilizan en la **IA generativa** incluyen:

- 1. Redes Generativas Antagónicas (GANs):** Dos redes neuronales compiten entre sí: una red generadora crea contenido, y una red discriminadora evalúa si ese contenido es real o generado.  
**Aplicaciones:** generación de imágenes realistas, videos, rostros humanos sintéticos (*deepfakes*), etc.  
**Ejemplos:** StyleGAN, CycleGAN, DCGAN.
- 2. Modelos de Difusión (Diffusion Models):** Generan contenido a través de un proceso gradual de eliminación de ruido, partiendo de ruido puro hasta llegar a una imagen clara y definida.  
**Aplicaciones:** generación de imágenes y arte de alta calidad, edición de imágenes.  
**Ejemplos:** Stable Diffusion, DALL-E 2, Imagen.

# IA Generativa

Los principales modelos de **Machine Learning (ML)** que se utilizan en la **IA generativa** incluyen:

4. **Transformers:** Utilizan el mecanismo de atención para entender relaciones entre los datos (texto, imágenes, audio) y generar resultados muy coherentes y contextualizados.

**Aplicaciones:** generación de textos largos, traducción automática, generación de código, subtitulado, resumen automático.

**Ejemplos:** GPT, BERT (más enfocado en clasificación y comprensión, aunque influyente en generación), T5, Codex.

5. **Autoencoders Variacionales (VAE):** Codifican datos de entrada en un espacio comprimido (espacio latente) y luego decodifican estos datos comprimidos para generar nuevas muestras similares al original.

**Aplicaciones:** generación de imágenes, reducción de dimensionalidad, creación de contenido original basado en variaciones controladas.

**Ejemplos:** VAE, Conditional VAE.

# HISTORIA DEL PROCESAMIENTO DE LENGUAJE NATURAL

Ignacio Gómez  
2025

# Historia del PLN

1. **Década de 1950-1960: Primeros Pasos.** En esta época se crean los primeros programas informáticos capaces de realizar tareas simples relacionadas con el lenguaje natural.
  - **1950: Alan Turing** propone el famoso **Test de Turing**, estableciendo criterios para determinar si una máquina puede considerarse inteligente a través de su uso del lenguaje natural.
  - **1954: Experimento Georgetown-IBM:** Traducción automática del ruso al inglés, considerada una de las primeras demostraciones exitosas de traducción automática, aunque con capacidades muy limitadas.
2. **Década de 1960-1970: Primeros intentos con reglas lingüísticas:** Se centran esfuerzos en crear sistemas basados en reglas lingüísticas y gramáticas formales.
  - **1960s: Joseph Weizenbaum** desarrolla **ELIZA**, considerado el primer chatbot. Usaba reglas simples para simular una conversación con un psicoterapeuta.
  - **1970s:** Aparecen gramáticas formales como las Gramáticas libres de contexto, y los primeros intentos de parsers sintácticos para analizar el lenguaje.

# Historia del PLN

3. **1980s – Surgimiento de enfoques estadísticos:** Se evidencian las limitaciones de las reglas estrictas. Surgen enfoques probabilísticos.
  - Desarrollo de **modelos probabilísticos** y estadísticas aplicadas al lenguaje.
  - Se crean los primeros algoritmos estadísticos para reconocimiento del habla y traducción automática.
4. **1990s: Boom de los modelos estadísticos y corpus lingüísticos:** Durante esta década, el NLP da un salto significativo, impulsado por la disponibilidad de grandes corpus digitales y mejoras computacionales.
  - **IBM Watson** (IBM) y el **modelo oculto de Markov (HMM)** revolucionan la traducción automática y el reconocimiento de voz.
  - Desarrollo de corpus lingüísticos masivos (como Penn Treebank).
  - Introducción de algoritmos para etiquetado gramatical (**POS tagging**) y análisis sintáctico automático (**parsing**).
5. **2000-2010: Machine Learning y NLP moderno:** Con el auge del aprendizaje automático supervisado y no supervisado, el NLP comienza a avanzar rápidamente.
  - Aparición de técnicas como BoW (Bag of words) o N-grams y técnicas de incrustación de palabras (**embeddings**).
  - Comienzan a popularizarse algoritmos como **Support Vector Machines (SVM)** y posteriormente métodos basados en redes neuronales profundas.

# Historia del PLN



**IBM Watson en Jeopardy**



# Historia del PLN

## 6. 2010-2017: El avance de Deep Learning y redes neuronales

- Llegan las **redes neuronales recurrentes (RNN)** y posteriormente las **LSTM** (Long Short-Term Memory), permitiendo mejor manejo del contexto y secuencias largas.
- Aparece **Word2Vec** (2013) y luego **GloVe**, modelos para representar palabras en vectores (embeddings).
- El NLP comienza a resolver tareas más complejas: análisis de sentimientos avanzado, traducción automática de calidad superior, etc.

## 7. 2017-Actualidad: La revolución de los Transformers y Modelos de Lenguaje Grandes: En 2017, un momento histórico marca un antes y un después en NLP: Se presenta la arquitectura **Transformer** en el artículo "*Attention is All You Need*". Esto cambia radicalmente la forma en que se procesan textos, permitiendo entrenar modelos en grandes corpus con resultados extraordinarios.

- **2018: BERT** (Google) trae comprensión textual bidireccional profunda.
- **2018-2019:** GPT-2 y posteriormente GPT-3 de OpenAI ofrecen generación de texto sorprendentemente realista.
- **2022-2024:** Explosión de modelos generativos como **ChatGPT, GPT-4, Claude, LLaMA**, que logran un rendimiento similar al humano en muchas tareas lingüísticas.

# PROCESAMIENTO DE LENGUAJE NATURAL

Ignacio Gómez  
2025

# Procesamiento de Lenguaje Natural

- El **procesamiento de lenguaje natural** (PLN o NLP) es el área de la inteligencia artificial que permite que una máquina procese, analice, transforme o genere lenguaje humano.
- En ingeniería aeroespacial, el lenguaje aparece en muchas fuentes:

| Fuente textual            | Ejemplo de uso                                              |
|---------------------------|-------------------------------------------------------------|
| Reportes de incidentes    | Clasificar eventos de seguridad operacional                 |
| NOTAMs                    | Extraer restricciones, pistas cerradas o cambios operativos |
| Manuales de mantenimiento | Buscar instrucciones técnicas                               |
| Requisitos de sistema     | Detectar ambigüedad, duplicados o conflictos                |
| Comunicaciones ATC        | Transcripción, clasificación o detección de riesgo          |
| Bitácoras de vuelo        | Resumen automático y detección de anomalías                 |
| Informes de ensayo        | Extracción de resultados, condiciones y conclusiones        |

# Problema central del NLP

- El lenguaje humano es difícil para una máquina porque es:
  - **Ambiguo**: “banco” puede ser una entidad financiera o un asiento.
  - **Contextual**: “clear” puede significar “autorizado”, “despejado” o “claro”.
  - **Variable**: pilotos, ingenieros y controladores usan abreviaturas, jerga y estilos distintos.
  - **Dependiente del dominio**: “stall”, “yaw”, “trim”, “flare” o “hold short” tienen significados técnicos.
  - **Ruidoso**: reportes reales pueden contener errores, abreviaturas, frases incompletas o formatos inconsistentes.
- En aeroespacial, esta dificultad aumenta porque muchos textos son **críticos para la seguridad**. Un sistema de NLP no solo debe “funcionar bien”; también debe ser trazable, evaluable y supervisado por expertos humanos.

# Tareas típicas del procesamiento de lenguaje

Clasificación

Generación

Resumen

Sentimiento

Temas

Traducción

Intención

Normalización

Búsqueda semántica

Entidades

Asistentes

Pregunta-respuesta

## Idea central

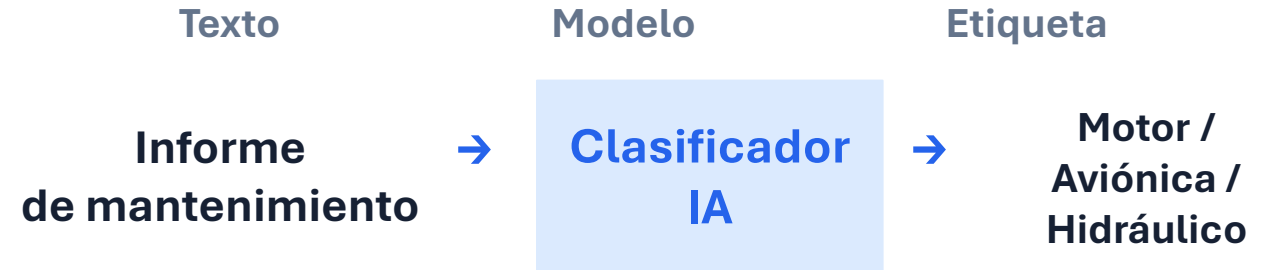
**Convertir texto técnico en datos útiles para buscar, decidir y automatizar con trazabilidad.**

# 1 - Clasificación de texto

---

## Objetivo

Asignar una etiqueta a un documento, mensaje o fragmento técnico.



## Ejemplo de entrada

“Hydraulic pressure dropped below nominal during approach.”



## Salida esperada

Sistema: hidráulico

Severidad: alta

Fase: aproximación

## 2 - Análisis de polaridad y sentimiento

---

### Más que “positivo / negativo”

En textos técnicos puede estimar tono operativo: alarma, incertidumbre, confianza o frustración.

#### Ejemplo

“The new flight planning software is unreliable during route optimization.”

**Polaridad: negativo**

**Sentimiento: frustración**

### Tono detectado

**Alarma**

riesgo, urgencia, fallo crítico

**Incertidumbre**

posible, intermitente, no confirmado

**Confianza**

estable, nominal, verificado

**Frustración**

usabilidad, retrasos, repetición de errores

# 3 - Detección de intención (intents)

## Pregunta clave

### ¿Qué quiere hacer el usuario?

La intención permite activar la acción correcta en un asistente técnico.

#### Usuario

“Show me the emergency checklist for cabin depressurization.”

#### Intención detectada

**consultar\_procedimiento\_emergencia**

## Catálogo de intenciones

**Buscar  
procedimiento**

**Resumir  
informe**

**Definir  
término**

**Comparar  
datos**

**Consultar  
normativa**

Cada intención conecta lenguaje natural con una función del sistema.



## 4 - Reconocimiento de entidades nombradas (NER)

---

**NER detecta elementos importantes dentro del texto.**

En aeroespacial conviene definir entidades propias del dominio.

Texto técnico

“During ascent, the Falcon 9 first stage experienced abnormal vibration near Max-Q.”

### Entidades extraídas

**Fase de vuelo: ascent**

**Vehículo: Falcon 9**

**Componente: first stage**

**Evento: abnormal vibration**

**Condición: Max-Q**

# 5 - Resumen automático

---

## Objetivo

**Reducir documentos largos sin perder los hechos técnicos esenciales.**

**Documento completo**



**Modelo de resumen**



**Resumen trazable**

## Ejemplo de uso

Informe de anomalía térmica de 20 páginas → síntesis ejecutiva para revisión de misión.

**Extractivo**

selecciona frases del documento; menor riesgo de inventar.

**Abstractivo**

redacta frases nuevas; más compacto, pero exige verificación.

# 6 - Traducción automática

---

## Objetivo

**Convertir documentación técnica entre idiomas manteniendo precisión terminológica.**

## Ejemplo

“Check the pressure relief valve before engine start.”

“Compruebe la válvula de alivio de presión antes del arranque del motor.”

## Riesgo de ambigüedad

**stall**

puede significar pérdida aerodinámica, parada, bloqueo o calado según el contexto.

**Glosarios**

**Memoria de traducción**

**Revisión experta**

**Terminología estándar**

# 7 - Búsqueda semántica

## Objetivo

Encontrar documentos por significado, aunque no usen las mismas palabras.

### Consulta

“problemas de sobrecalentamiento en turbina”

Puede recuperar documentos sobre:

excessive turbine inlet temperature

thermal anomaly in engine section

Consulta



Embedding



Documentos cercanos

Fuentes típicas  
manuales técnicos · informes de fallos · lecciones aprendidas · normativas · papers científicos

Ventaja  
Reduce dependencia de palabras exactas y ayuda a encontrar incidentes similares.

Cuidado

Requiere validar que el resultado sea técnicamente relevante, no solo parecido semánticamente.

# 8 - Pregunta-respuesta

---

## Objetivo

**Responder preguntas usando documentos técnicos como fuente de verdad.**

## Pregunta

“What is the maximum allowable cabin pressure differential?”

## Respuesta esperada

Valor exacto + referencia al manual o procedimiento.



# 9 - Generación de texto

---

## Objetivo

**Producir texto nuevo a partir de datos, instrucciones o documentos.**

### Datos de entrada

Componente: APU  
Fallo: arranque fallido  
Condición: baja temperatura exterior  
Acción: inspección de combustible

### Texto generado

“Se registró un fallo de arranque de la APU en condiciones de baja temperatura. Se recomienda inspeccionar el sistema de combustible asociado.”

## Aplicaciones

Borradores de informes

Procedimientos iniciales

Respuestas técnicas

Documentación base

Narrativas desde datos

### Control necesario

En documentación de seguridad, el texto generado debe revisarse y compararse con datos fuente.

# 10 - Detección de temas o keywords

---

## Objetivo

**Descubrir asuntos recurrentes en grandes colecciones de documentos.**

## Base de ejemplo

100 000 informes de mantenimiento con texto libre.

El sistema agrupa palabras y documentos que tienden a aparecer juntos.

### Tema 1

hydraulic · pump · actuator · leak · landing gear

### Tema 2

vibration · N2 · EGT · compressor · climb

### Tema 3

battery · temperature · eclipse · thermal · limit

### Tema 4

sensor · calibration · intermittent · avionics

## Uso

Detectar patrones de fallos, tendencias emergentes y áreas que requieren análisis de causa raíz.

# 11 - Corrección y normalización técnica

---

## Objetivo

**Corregir errores y unificar la forma en que se nombran conceptos técnicos.**

## Corrección

“Engin temp exceded limits durin climb.”



“Engine temperature exceeded limits during climb.”

## Normalización

left engine

engine no. 1

ENG 1



**ENGINE\_1**



# 12 - Diálogo y asistentes conversacionales

---

## Objetivo

**Combinar varias tareas de PLN para interactuar con usuarios técnicos.**

## Ejemplo de diálogo

Ingeniero: “Busca informes de vibraciones durante ascenso.”

Asistente: “Encontré 34 informes; la mayoría se asocia al motor izquierdo entre FL180 y FL250.”

Ingeniero: “Resume los tres más graves.”

Intención

Búsqueda  
semántica

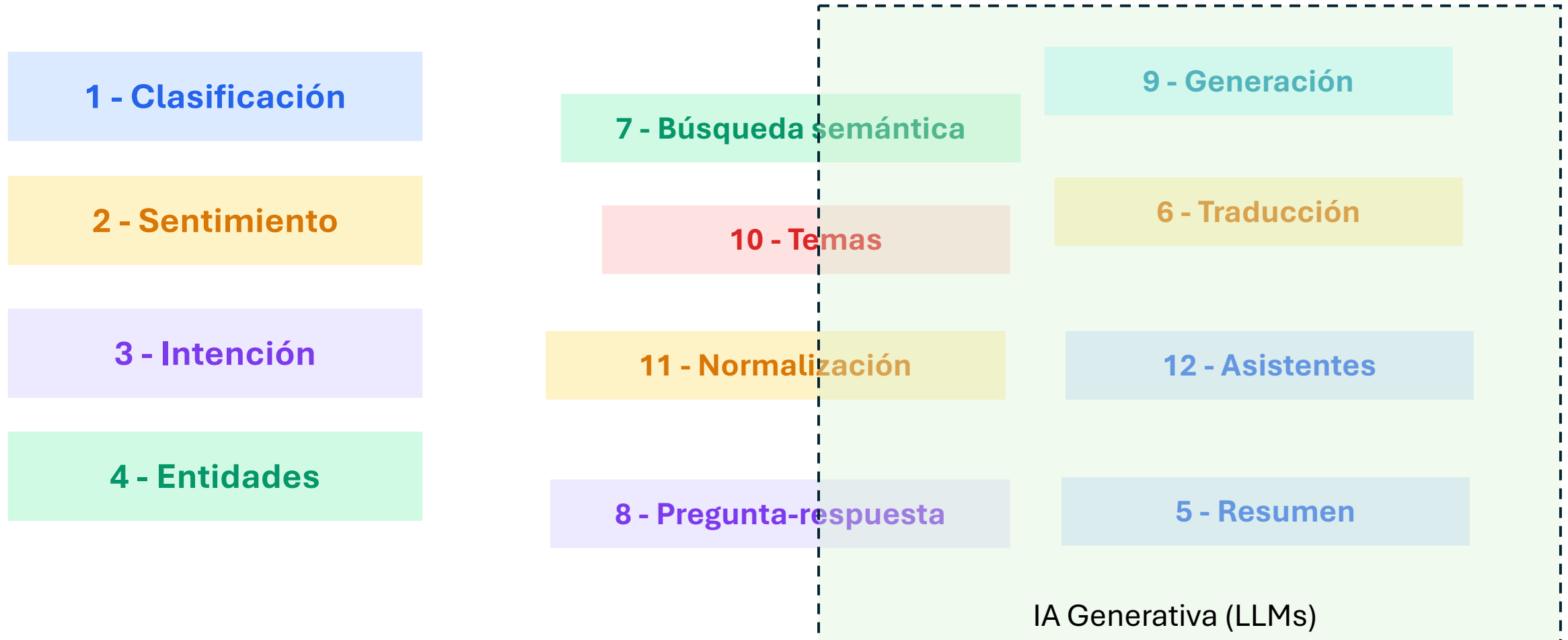
Pregunta-  
respuesta

Resumen

Generación

**Un asistente útil no es una sola tarea:  
es una orquestación.**

# Tareas típicas del procesamiento de lenguaje

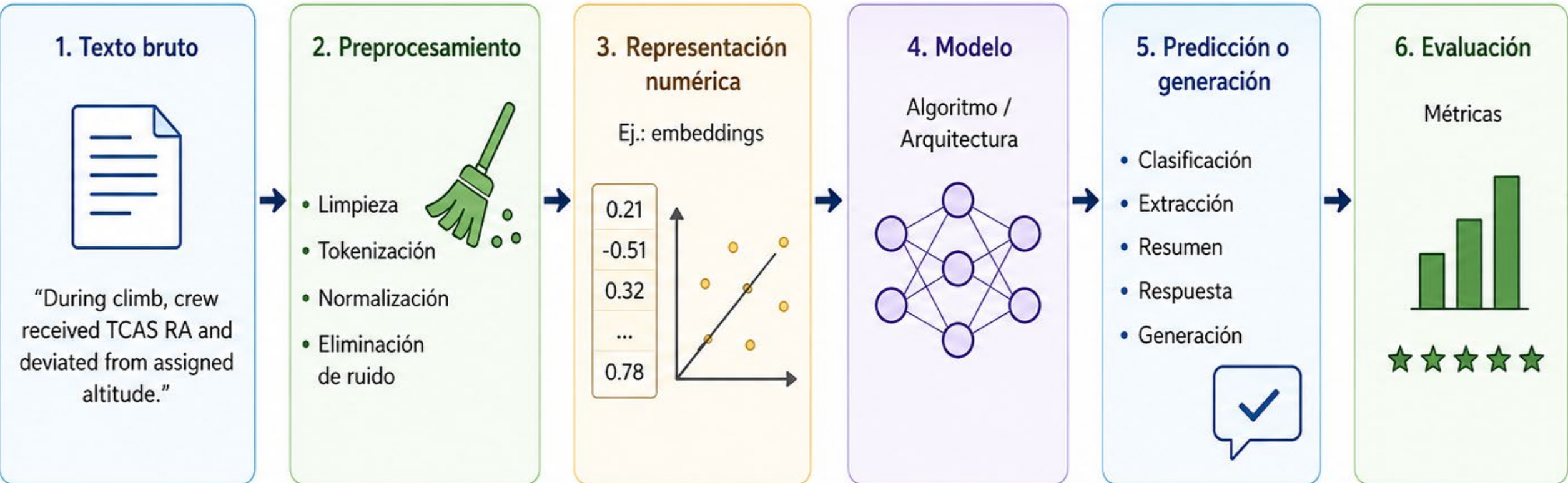


# OpenAI

| Año  | Ingresos                                                                                                                                    | Beneficio o pérdida                                                                                            |
|------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| 2021 | No hay cifra pública fiable. Actividad comercial aún pequeña, antes de ChatGPT.                                                             | No disponible.                                                                                                 |
| 2022 | ~ <b>28 M\$</b> de ingresos, según The Information/Fortune.                                                                                 | ~- <b>540 M\$</b> . ( <a href="#">The Information</a> )                                                        |
| 2023 | El run-rate anualizado superó <b>1.600 M\$</b> a finales de 2023; OpenAI había esperado inicialmente solo 200 M\$ para ese año.             | Pérdida no auditada; estimaciones externas sitúan 2023 aún claramente en negativo. ( <a href="#">Reuters</a> ) |
| 2024 | ~ <b>3.700 M\$</b> de ingresos esperados/estimados; a cierre de 2024 OpenAI ya generaba ~ <b>1.000 M\$ por trimestre</b> .                  | ~- <b>5.000 M\$</b> . ( <a href="#">Fortune</a> )                                                              |
| 2025 | Primer semestre: <b>4.300 M\$</b> ; objetivo anual citado: ~ <b>13.000 M\$</b> ; run-rate anualizado a final de 2025: > <b>20.000 M\$</b> . | Pérdida / cash burn estimado: ~- <b>8.000 a -8.500 M\$</b> , según reportes. ( <a href="#">Reuters</a> )       |

# FLUJO GENERAL DE UN SISTEMA NLP

*Del texto bruto a la información útil*

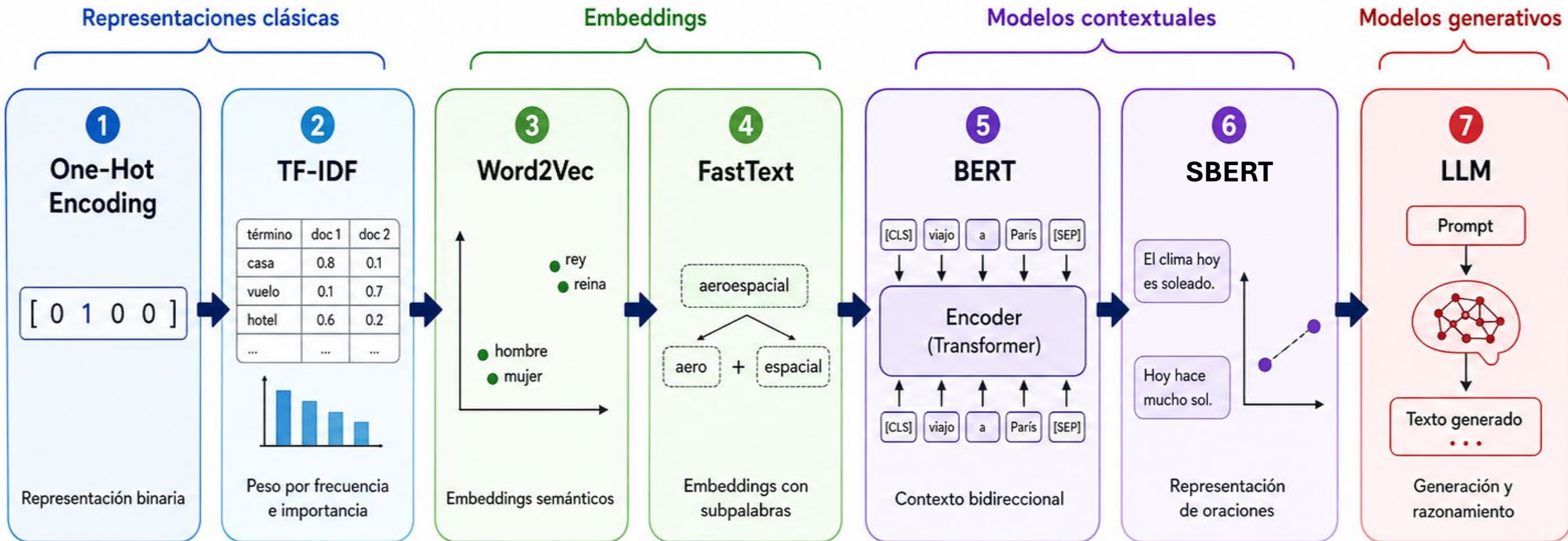


# Preproceso

- **Limpieza:** Incluye eliminar caracteres extraños, corregir codificación, normalizar mayúsculas y minúsculas, o separar campos estructurados.
- **Tokenización:** La tokenización divide el texto en unidades: palabras, subpalabras o caracteres.
- **Lematización y stemming:** Estas técnicas reducen palabras a una forma base, por ejemplo, todas las formas verbales al infinitivo.

# EVOLUCIÓN DE REPRESENTACIONES Y MODELOS DE NLP

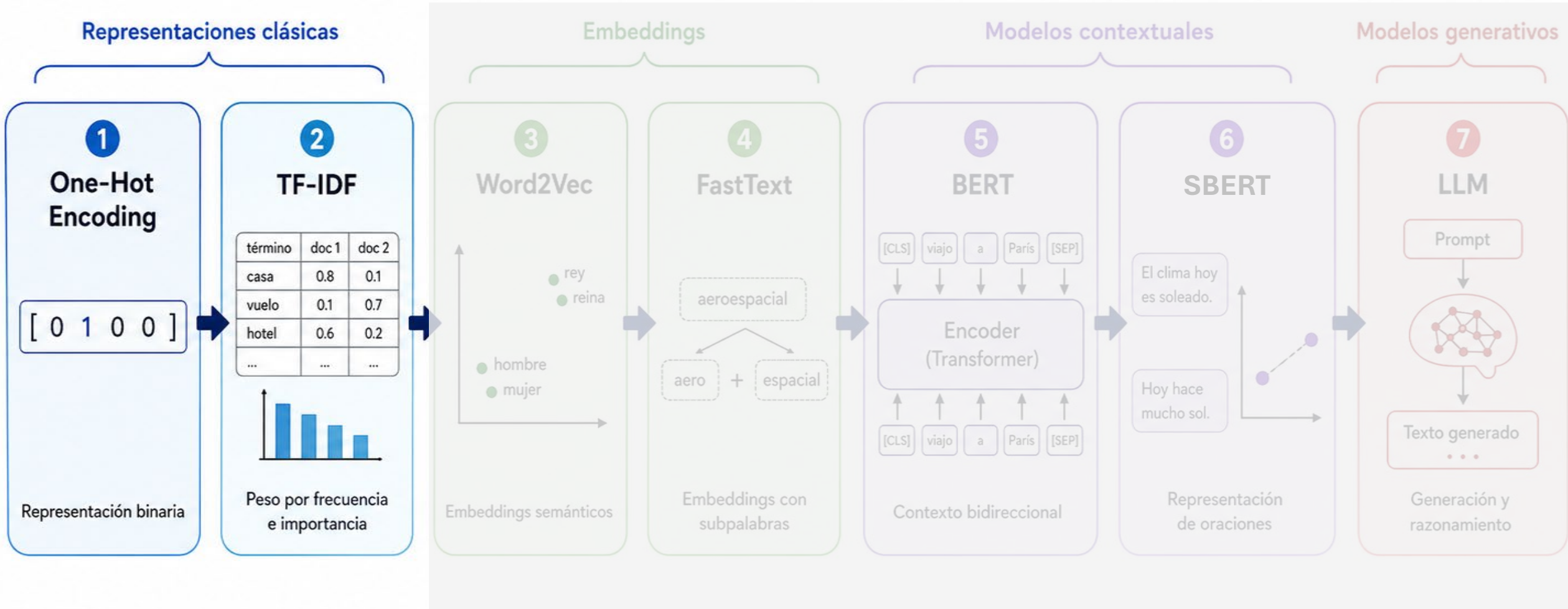
*De representaciones discretas a modelos de lenguaje avanzados*





# EVOLUCIÓN DE REPRESENTACIONES Y MODELOS DE NLP

*De representaciones discretas a modelos de lenguaje avanzados*



# Word embeddings

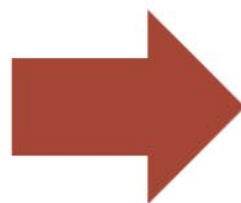
- El concepto central de los word embeddings (o vectores de palabras) es que cada palabra utilizada en un lenguaje puede ser representada por un conjunto de números reales (un vector).
- Los word embeddings son vectores N-dimensionales que intentan capturar el significado y el contexto de las palabras a través de sus valores. Para que sea realmente útil, el conjunto de vectores de un vocabulario debe capturar el significado de las palabras, las relaciones entre ellas y el contexto en el que se utilizan de forma natural.
- Características clave de un buen modelo:
  - **Unicidad:** Cada palabra tiene un word embedding único.
  - **Multidimensionalidad:** En un buen modelo, la longitud de estos vectores oscila entre 50 y 500 dimensiones.
  - **Captura de significado:** El vector logra representar la esencia o el "significado" de la palabra que codifica.
  - **Similitud semántica:** Las palabras que tienen significados similares terminan con valores de vector (embeddings) muy parecidos entre sí.



# Word embeddings

- La forma más sencilla de hacerlo es con one hot encoding del vocabulario completo, pero esto no cumple las características deseadas:

Vocabulary:  
Man, woman, boy,  
girl, prince,  
princess, queen,  
king, monarch



|          | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|----------|---|---|---|---|---|---|---|---|---|
| man      | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| woman    | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| boy      | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| girl     | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| prince   | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| princess | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| queen    | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| king     | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| monarch  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

Each word gets  
a 1x9 vector  
representation

# TF-IDF

- TF-IDF significa **Term Frequency – Inverse Document Frequency**, en español: frecuencia de término – frecuencia inversa de documento. Es una técnica usada en procesamiento de lenguaje natural, buscadores y minería de texto para medir cómo de importante es una palabra dentro de un documento, comparándola con una colección de documentos.
- **TF** mide cuántas veces aparece una palabra en un documento.  
**IDF** reduce el peso de palabras demasiado comunes en muchos documentos, como “el”, “la”, “de”, “y”.  
**TF-IDF** da más valor a palabras que aparecen mucho en un documento, pero no en todos los documentos.
- Ver cuaderno: Embeddings\_NLP Puntos 1 y 2

# EVOLUCIÓN DE REPRESENTACIONES Y MODELOS DE NLP

*De representaciones discretas a modelos de lenguaje avanzados*

## Representaciones clásicas

1

One-Hot  
Encoding

[ 0 1 0 0 ]

Representación binaria

2

TF-IDF

| término | doc 1 | doc 2 |
|---------|-------|-------|
| casa    | 0.8   | 0.1   |
| vuelo   | 0.1   | 0.7   |
| hotel   | 0.6   | 0.2   |
| ...     | ...   | ...   |



Peso por frecuencia  
e importancia

## Embeddings

3

Word2Vec



Embeddings semánticos

4

FastText

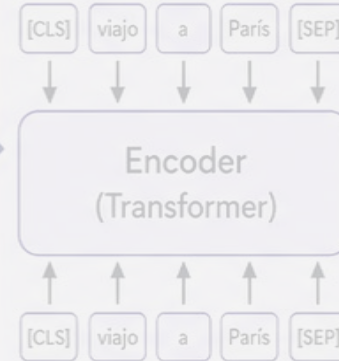


Embeddings con  
subpalabras

## Modelos contextuales

5

BERT



Contexto bidireccional

6

SBERT

El clima hoy  
es soleado.

Hoy hace  
mucho sol.

Representación  
de oraciones

## Modelos generativos

7

LLM

Prompt



Texto generado  
...

Generación y  
razonamiento

# Word embeddings

- Existen algunos problemas con el enfoque one-hot para la codificación:
  - Crecimiento lineal de dimensiones: El número de dimensiones (columnas en este caso) aumenta de forma lineal a medida que añadimos palabras al vocabulario. Para un vocabulario de 50,000 palabras, cada palabra se representa con 49,999 ceros y un único valor "uno" en la posición correspondiente.
  - Dispersión de datos: La matriz de representación es muy dispersa (sparse), ya que está compuesta principalmente por ceros.
  - Falta de relación semántica: No existe información compartida entre las palabras ni similitudes entre términos parecidos. Todas las palabras se encuentran a la misma "distancia" entre sí en el espacio de representación de 9 dimensiones.

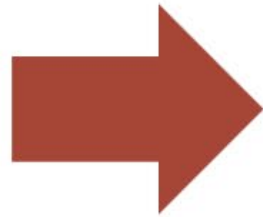
# Palabras similares

- Similares: No son sinónimos necesariamente, pero comparten algún tipo de relación entre ellas. Esta similaridad puede referirse a categorías muy diversas:
  - Perro – Gato → Animales de compañía
  - Vaca – Caballo → Animales domésticos
  - León – Gato → Felinos
  - Ballena – Submarino → Relacionados con el mar
  - Bisturí – Cirujano → Aparecen en entornos sanitarios
  - Bisturí – Cuchillo → Herramientas con filo
  - Claro – Oscuro → Determinan las características de luz
  - Alegría – Entusiasmo → Tienen valencia positiva
- Dos palabras son similares si se pueden sustituir entre si, o si frecuentemente aparecen próximas en frases.

# Custom Encoding

## Vocabulary:

Man, woman, boy,  
girl, prince,  
princess, queen,  
king, monarch



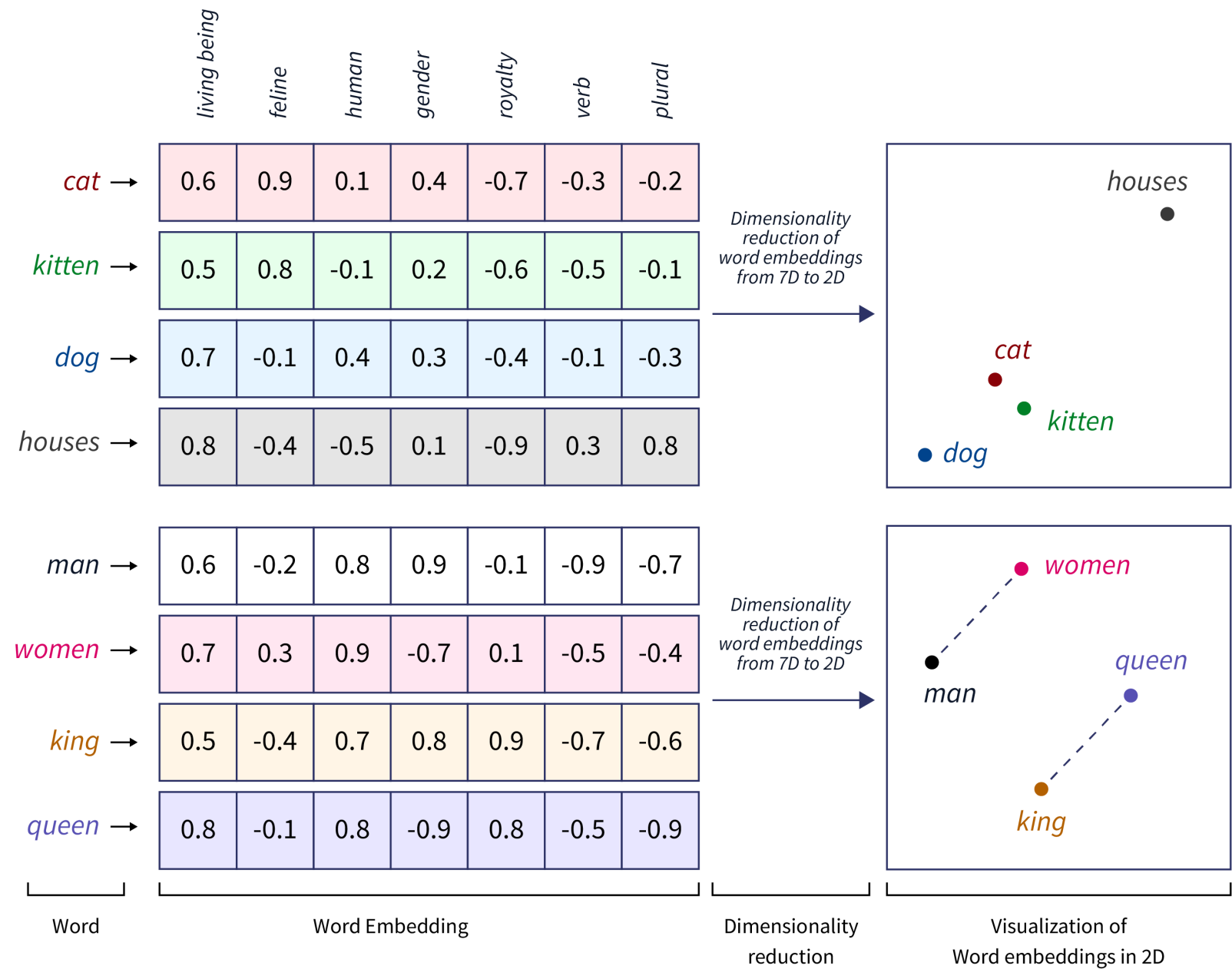
|          | Femininity | Youth | Royalty |
|----------|------------|-------|---------|
| Man      | 0          | 0     | 0       |
| Woman    | 1          | 0     | 0       |
| Boy      | 0          | 1     | 0       |
| Girl     | 1          | 1     | 0       |
| Prince   | 0          | 1     | 1       |
| Princess | 1          | 1     | 1       |
| Queen    | 1          | 0     | 1       |
| King     | 0          | 0     | 1       |
| Monarch  | 0.5        | 0.5   | 1       |

Each word gets a  
1x3 vector

Similar words...  
similar vectors

Pero, con un número grande de palabras se hace imposible

# Embeddings



# ¿Como de similares son dos vectores?

- Producto escalar:

$$\text{dot product}(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i = v_1 w_1 + v_2 w_2 + \dots + v_N w_N$$

- Tiende a ser grande cuando los dos vectores tienen valores grandes en la misma dimension.
- Podría ser una buena métrica.



# Problema con el producto escalar

- Favorece a los vectores grandes
- Es mayor si un vector es más grande (tiene mayores valores en varias dimensiones), es decir mayor modulo:

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^N v_i^2}$$

- Las palabras más frecuentes tienen vectores con mayor modulo porque tienen similitudes con mayor número de palabras, así que las distancias se falsean.

# Alternativa: cosine similarity

$$\text{cosine}(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

# ¿como obtener la similitud?

Supongamos que vemos estas frases:

- El ong choi es delicioso salteado con ajo.
- El ong choi es excelente sobre arroz.
- Hojas de ong choi con salsas saladas.

Y también has visto estas:

- ...espinacas salteadas con ajo sobre arroz.
- Los tallos y hojas de acelga son deliciosos.
- Col rizada y otras verduras de hoja verde saladas.

Conclusión:

El ong choi es una verdura de hoja verde como la espinaca, la acelga o la col rizada. Podríamos concluir esto basándonos en palabras como "hojas", "delicioso" y "salteado".

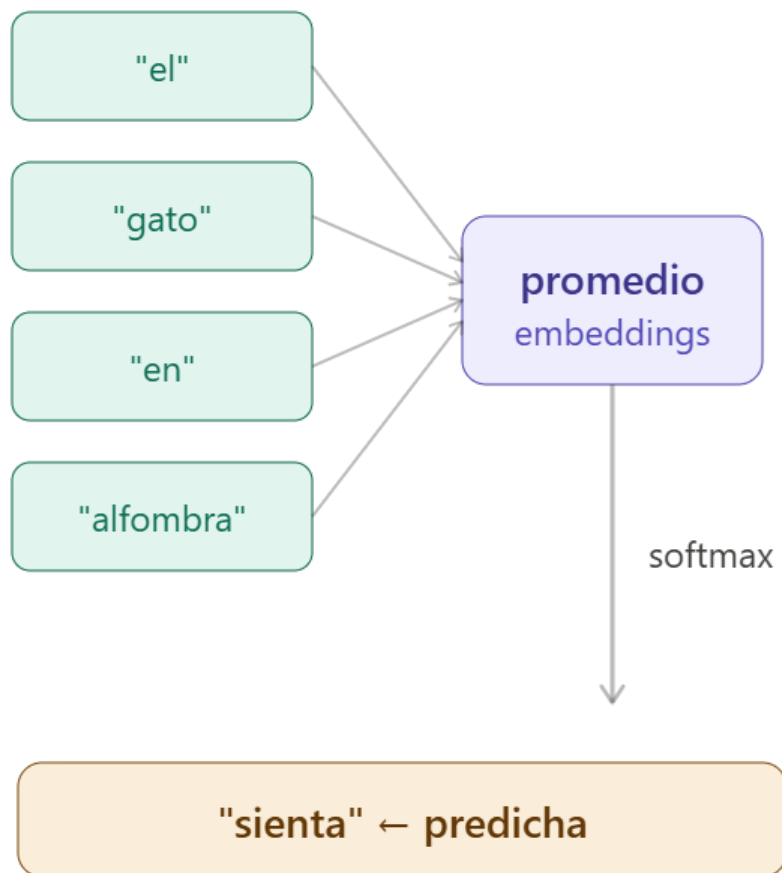


# Word2vec (2013)

- Word2Vec es una familia de modelos neuronales simples cuyo objetivo es aprender una representación en embeddings con información semántica.
- Word2Vec suele entrenarse con una de dos arquitecturas:
  - **CBOW**, *Continuous Bag of Words*: intenta predecir la palabra central usando las palabras de alrededor.
  - **Skip-Gram**: hace lo contrario: usa la palabra central para predecir las palabras vecinas.

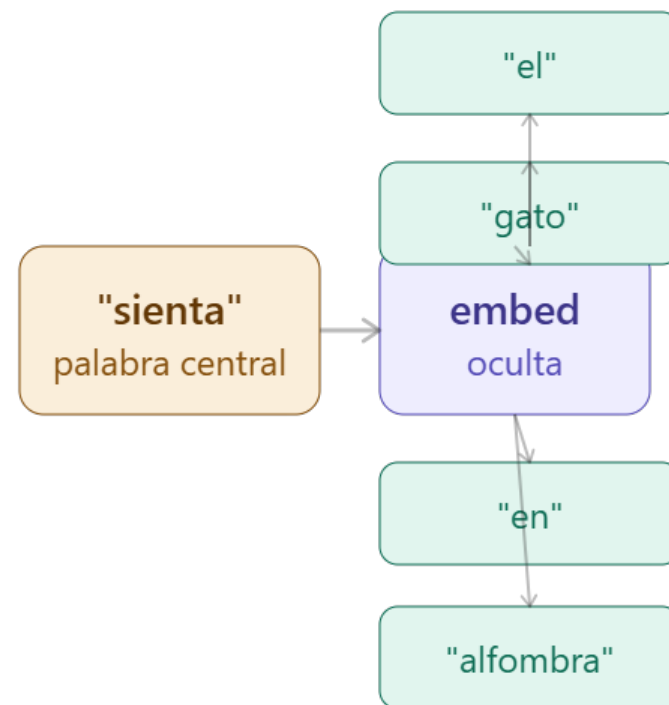
## CBOW

contexto → palabra central



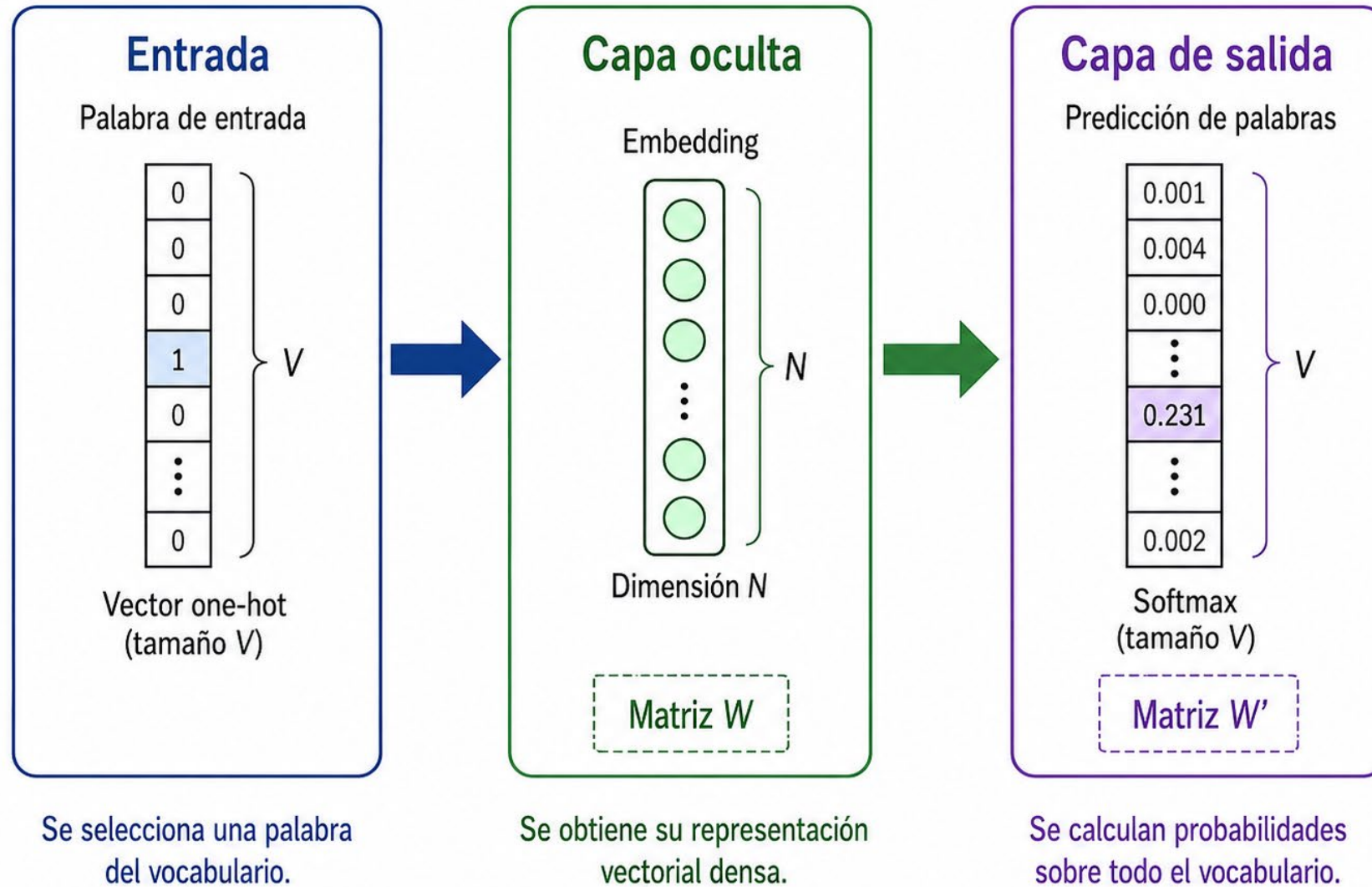
## Skip-gram

palabra central → contexto



Frase: "el gato **sienta** en alfombra" — ventana de contexto = 2

# Arquitectura interna de Word2Vec



Una vez entrenado Word2Vec, normalmente ya no usamos la red completa. Usamos la matriz de embeddings.

# Entrenamiento

- **Paso 1: recopilar corpus:**

- manuales de mantenimiento

- informes de vuelo

- papers técnicos

- documentación de sistemas

- informes de anomalías

- telemetría textual

- procedimientos de operación

- normativa aeronáutica

- documentos de diseño preliminar

- La calidad del corpus es crítica. Word2Vec aprende relaciones estadísticas, por lo que el modelo será tan bueno como el lenguaje que vea.

# Entrenamiento

- **Paso 2: preprocesar texto**
- Normalmente se realizan algunas operaciones:
  - minúsculas
  - tokenización
  - eliminación opcional de puntuación
  - detección de expresiones compuestas
  - filtrado de palabras muy raras
  - filtrado opcional de stopwords

En textos aeroespaciales conviene tener cuidado con unidades, símbolos y acrónimos:



# Entrenamiento

- **Paso 3: crear pares de entrenamiento:** Con una ventana de contexto se generan ejemplos.
- Texto: el cohete alcanza velocidad orbital
- Con ventana 2 y Skip-Gram:
  - cohete → el
  - cohete → alcanza
  - alcanza → el
  - alcanza → cohete
  - alcanza → velocidad
  - velocidad → cohete
  - velocidad → alcanza
  - velocidad → orbital

Ver cuaderno: [Embeddings\\_NLP](#) Puntos 3 y 4

# Donde obtener modelos pre-entrenados

- **Gensim** es una librería de Python usada en procesamiento de lenguaje natural para trabajar con grandes colecciones de texto.
- Sirve principalmente para:
  - Modelado de temas: descubrir temas ocultos en documentos, por ejemplo, con LDA o LSI.
  - Embeddings de palabras: entrenar o usar modelos como Word2Vec, FastText o Doc2Vec.
  - Búsqueda por similitud: comparar documentos o palabras según su significado.
  - Representación vectorial de texto: convertir palabras, frases o documentos en vectores numéricos.
- Ver cuaderno: [Embeddings\\_NLP Punto 5](#)

# EVOLUCIÓN DE REPRESENTACIONES Y MODELOS DE NLP

*De representaciones discretas a modelos de lenguaje avanzados*

## Representaciones clásicas

1

One-Hot  
Encoding

[ 0 1 0 0 ]

Representación binaria

2

TF-IDF

| término | doc 1 | doc 2 |
|---------|-------|-------|
| casa    | 0.8   | 0.1   |
| vuelo   | 0.1   | 0.7   |
| hotel   | 0.6   | 0.2   |
| ...     | ...   | ...   |



Peso por frecuencia  
e importancia

## Embeddings

3

Word2Vec



Embeddings semánticos

4

FastText

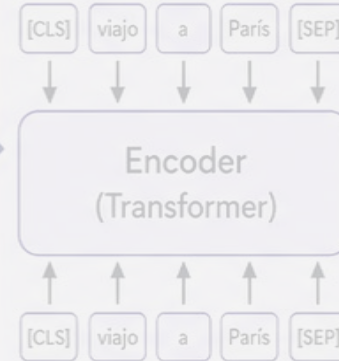


Embeddings con  
subpalabras

## Modelos contextuales

5

BERT



Contexto bidireccional

6

SBERT

El clima hoy  
es soleado.

Hoy hace  
mucho sol.

Representación  
de oraciones

## Modelos generativos

7

LLM

Prompt



Texto generado  
...

Generación y  
razonamiento

# FastText

- FastText es un modelo de embeddings de palabra desarrollado por Facebook AI Research. Se puede entender como una extensión de Word2Vec que mejora especialmente el tratamiento de palabras raras, palabras técnicas y palabras no vistas durante el entrenamiento.
- La idea clave es:
  - Word2Vec representa cada palabra como una unidad completa.
  - FastText representa cada palabra como una combinación de subpalabras.
- FastText no aprende solo un vector para cada palabra completa. También aprende vectores para fragmentos de palabras, normalmente llamados n-gramas de caracteres.

# FastText

- Por ejemplo, para la palabra: flutter
- FastText puede dividirla en fragmentos con n-gramas de longitud 3 a 6:
  - <fl, flu, flut, flutt, ...
  - lut, lutt, lutte, ...
  - tter, ter>
- Los símbolos < y > indican el inicio y el final de la palabra.
- El vector final de la palabra se calcula combinando el vector de la palabra completa y los vectores de sus subpalabras:

$\text{vector}(\text{"flutter"}) = \text{vector}(\text{"<fl"}) + \text{vector}(\text{"flu"}) + \text{vector}(\text{"lut"}) + \dots + \text{vector}(\text{"flutter"})$

# FastText

| Aspecto                                  | Word2Vec            | FastText                             |
|------------------------------------------|---------------------|--------------------------------------|
| <b>Representación</b>                    | Palabra completa    | Palabra + subpalabras                |
| <b>Maneja palabras no vistas</b>         | Mal                 | Mejor                                |
| <b>Captura morfología</b>                | Poco                | Mejor                                |
| <b>Coste computacional</b>               | Bajo                | Algo mayor                           |
| <b>Útil para términos técnicos raros</b> | Limitado            | Sí                                   |
| <b>Ejemplo</b>                           | flutter como unidad | flutter como combinación de n-gramas |

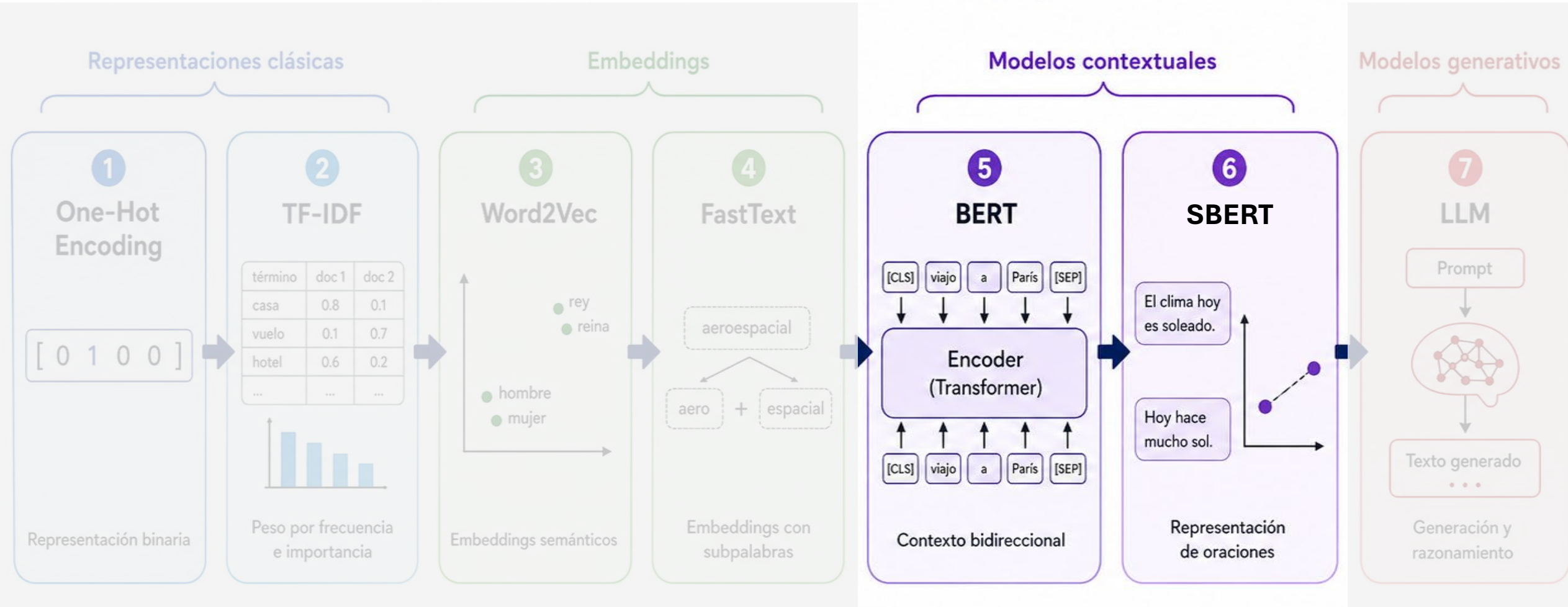
- Ver cuaderno: Embeddings\_NLP2 Punto 1

# Limitaciones

- Word2Vec tiene limitaciones importantes:
  - **Una palabra tiene un único vector:** La palabra “pitch” puede referirse a actitud de la aeronave, paso de hélice o tono acústico. Word2Vec clásico no distingue bien esos usos si asigna un único vector global.
  - **No representa directamente frases completas:** Word2Vec produce vectores de palabras, no de oraciones. Ignora parcialmente el orden. Las frases “el sensor detectó fallo” y “el fallo detectó sensor” contienen palabras similares, pero no significan lo mismo.
  - **No maneja bien negación y relaciones lógicas.** “No se detectó vibración crítica” y “se detectó vibración crítica” son muy parecidas a nivel léxico, pero opuestas en significado operativo.

# EVOLUCIÓN DE REPRESENTACIONES Y MODELOS DE NLP

*De representaciones discretas a modelos de lenguaje avanzados*





# BERT(2019)

- BERT significa **Bidirectional Encoder Representations from Transformers**.
- Su idea principal es que una palabra no debe representarse de forma aislada, sino según el contexto completo en el que aparece.
- El objetivo de BERT no es generar texto palabra por palabra, como haría un modelo autoregresivo tipo GPT. Su objetivo principal es entender y representar texto.
- La idea es añadir una capa de salida y hacer fine-tuning para tareas como:

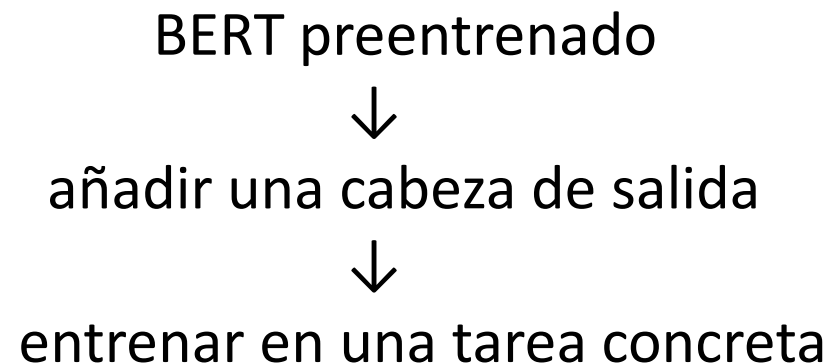
clasificación de texto  
respuesta a preguntas  
inferencia textual  
detección de similitud

análisis de sentimiento  
extracción de entidades  
búsqueda semántica

# BERT

| Modelo     | Capas | Dimensión oculta | Cabezas de atención | Parámetros aprox. |
|------------|-------|------------------|---------------------|-------------------|
| BERT-base  | 12    | 768              | 12                  | 110 M             |
| BERT-large | 24    | 1024             | 16                  | 340 M             |

- **Preentrenamiento:** BERT aprende lenguaje general a partir de grandes corpus de texto sin etiquetas manuales. Se considera entrenamiento autosupervisado: las etiquetas se construyen a partir del propio texto.
- **Fine-tuning:** Una vez preentrenado, BERT se adapta a una tarea concreta con ejemplos etiquetados:



Ver Hugging Face

# BERT

| Aspecto                      | FastText                                                        | BERT                                                                              |
|------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Tipo de modelo</b>        | Embeddings estáticos con subpalabras                            | Transformer contextual                                                            |
| <b>Unidad</b>                | Palabra + n-gramas de caracteres                                | Tokens/subtokens en contexto                                                      |
| <b>Vector de una palabra</b> | Casi fijo                                                       | Depende de la frase                                                               |
| <b>Palabras raras</b>        | Muy bien                                                        | Bien, por tokenización subword                                                    |
| <b>Contexto profundo</b>     | No                                                              | Sí                                                                                |
| <b>Coste computacional</b>   | Bajo                                                            | Alto                                                                              |
| <b>Uso típico</b>            | Línea base rápida, clasificación eficiente, vocabulario técnico | Comprensión contextual, QA, clasificación avanzada, embeddings de frase con SBERT |

# Embeddings de sentencia

- Una forma sencilla de pasar de palabras a frases es promediar los vectores de las palabras.
- Supongamos la frase:  
"se detectó flutter en el ala"
- Si tenemos vectores para:  
se, detectó, flutter, en, el, ala
- podemos construir:  
$$\text{embedding\_frase} = \text{media}(\text{embedding\_palabra\_1}, \dots, \text{embedding\_palabra\_n})$$
- En forma matemática:  
$$V_{\text{frase}} = (v_1 + v_2 + \dots + v_n) / n$$
- Este método es útil como línea base, pero es limitado. La media de palabras no sabe realmente qué palabra modifica a cuál, ni entiende si una frase niega una condición crítica.

# Sentence-BERT

**Sentence-BERT**, o **SBERT**, modifica BERT usando arquitecturas siamesas o de tripletas para obtener embeddings de frases que puedan compararse directamente mediante similitud coseno.

La idea es sencilla:

Frase A  $\rightarrow$  BERT/SBERT  $\rightarrow$  vector A

Frase B  $\rightarrow$  BERT/SBERT  $\rightarrow$  vector B

$\text{similitud} = \text{coseno}(\text{vector A}, \text{vector B})$

En vez de meter las dos frases juntas en el modelo cada vez, calculamos previamente un vector para cada frase. Después, la comparación es solo una operación vectorial.

# Sentence-BERT

Por ejemplo, para un sistema real de recuperación semántica en documentación aeroespacial:

1. Dividir documentos en fragmentos
2. Limpiar texto técnico
3. Generar embeddings con SBERT
4. Guardar embeddings en una base vectorial
5. Convertir la consulta del usuario en embedding
6. Buscar los vectores más cercanos
7. Mostrar los fragmentos recuperados

# Sentence-BERT

| Aspecto                    | Word2Vec promedio        | Sentence-BERT                      |
|----------------------------|--------------------------|------------------------------------|
| <b>Unidad básica</b>       | Palabra                  | Frase, oración o párrafo corto     |
| <b>Contexto</b>            | Limitado                 | Contextual                         |
| <b>Orden de palabras</b>   | Débil                    | Mejor capturado                    |
| <b>Negación</b>            | Débil                    | Mejor, aunque no perfecto          |
| <b>Polisemia</b>           | Mala en Word2Vec clásico | Mejor                              |
| <b>Búsqueda semántica</b>  | Posible, pero básica     | Mucho más adecuada                 |
| <b>Coste de inferencia</b> | Bajo                     | Medio                              |
| <b>Calidad semántica</b>   | Limitada                 | Alta para similitud y recuperación |

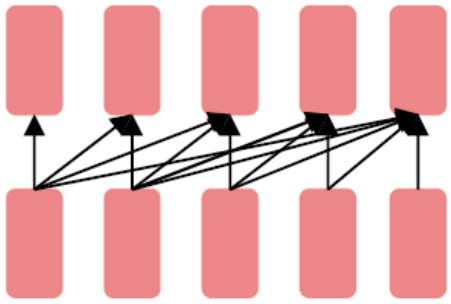
- Ver cuaderno: Embeddings\_NLP2 Punto 2
- Ver cuaderno: Embeddings\_NLP2 Punto 3 como aplicación práctica final
- Ver clasificacion\_autores

# LLM

Ignacio Gómez  
2025

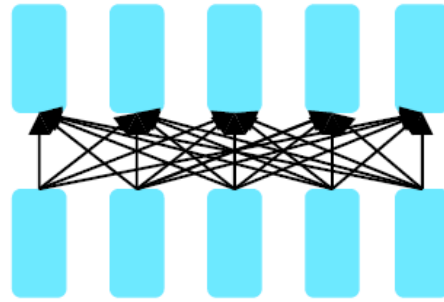


# Grandes modelos de lenguaje



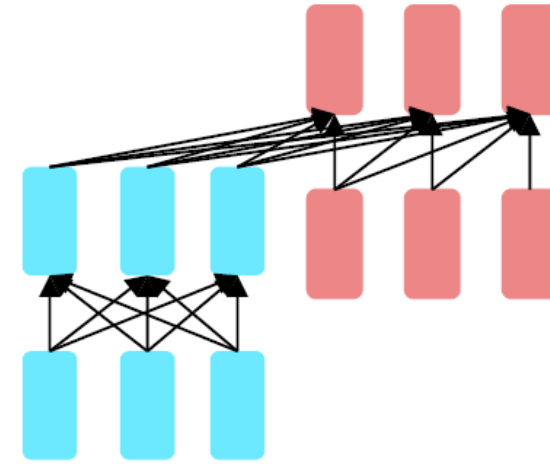
## Decoders

GPT, Claude,  
Llama  
Mixtral



## Encoders

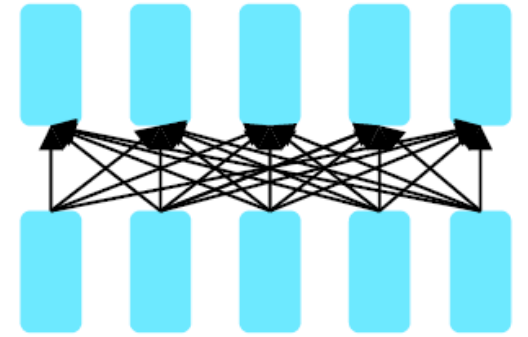
BERT family,  
HuBERT



## Encoder-decoders

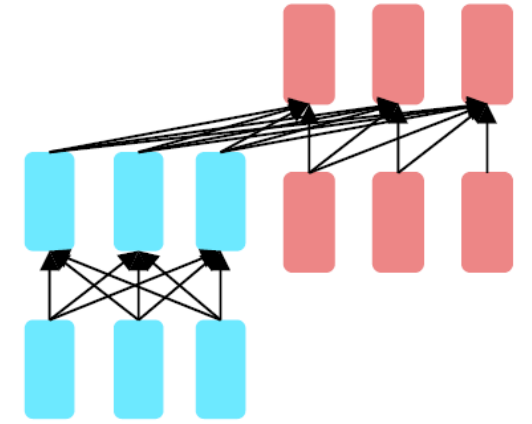
Flan-T5, Whisper

# Encoders



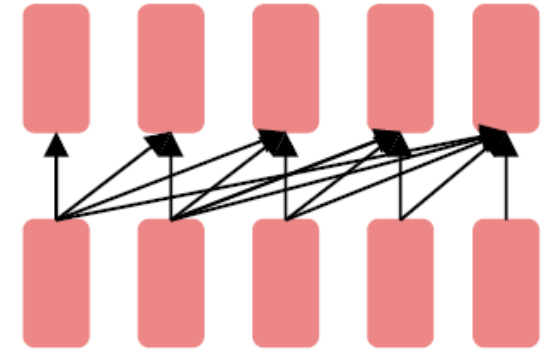
- Los más populares son los modelos de la familia BERT.
- Se entrenan prediciendo una palabra a partir de las palabras que la rodean por los dos lados.
- Normalmente, después se hace fine-tuning con datos supervisados para tareas de clasificación.

# Encoders-Decoders



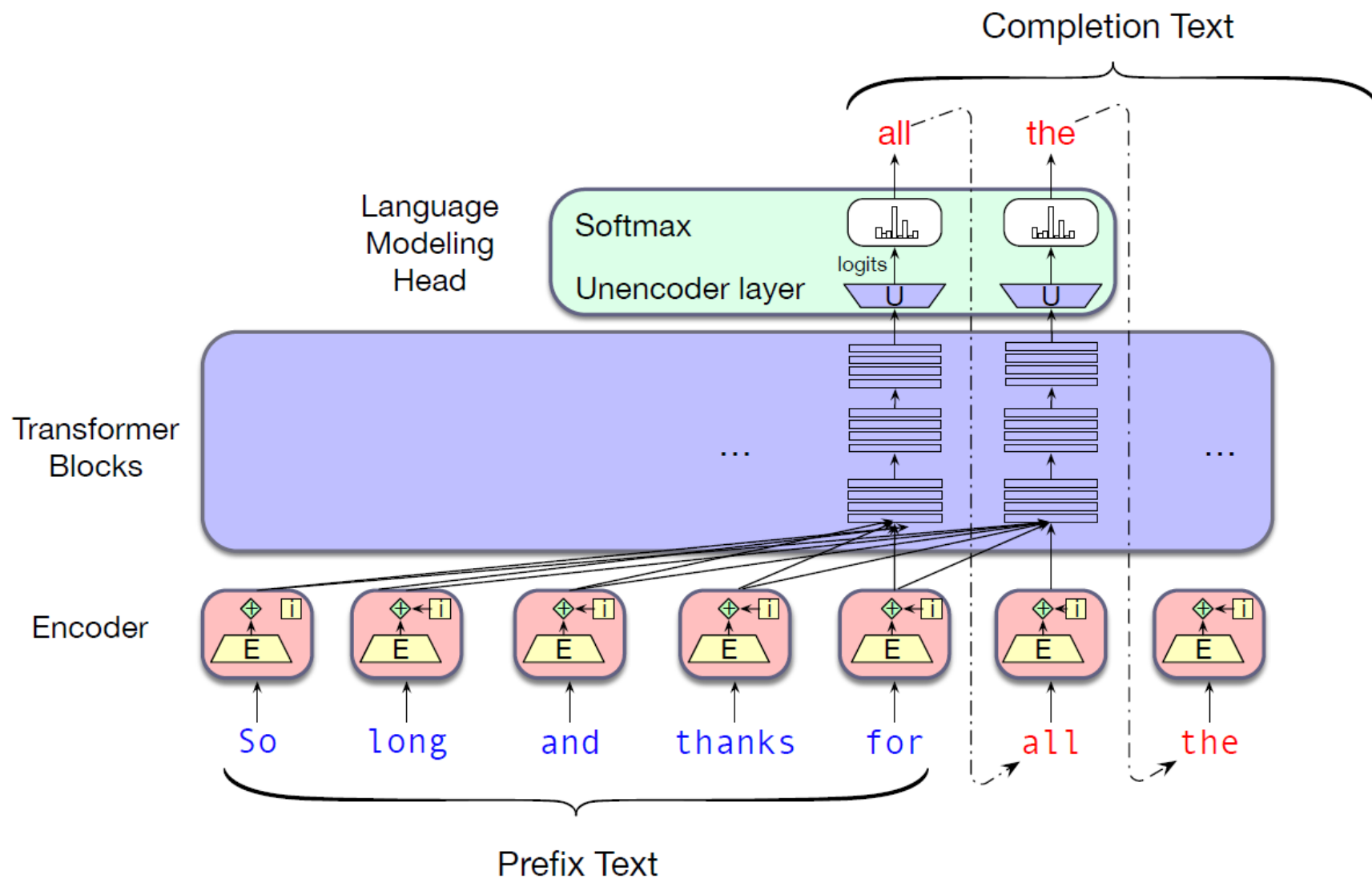
- Se entrenan para predecir una secuencia de salida a partir de otra de entrada.
- Las tareas típicas para las que se utilizan son:
  - Traducción automática.
  - Transcripción de voz a texto.

# Decoders



- Son los que se han hecho super populares a partir de la aparición de ChatGPT.
- Predicen palabras de izquierda a derecha.
- También se llaman:
  - LLMs Causales
  - LLMs Autoregresivos
  - LLMs de izquierda a derecha

# ¿Qué significa autoregresivo?



# Muchas tareas pueden ponerse de esta forma

- **Análisis de sentimientos:**

¿Cuál es la polaridad de la frase “Me gusta Monica Belucci”?

1. Pasamos al modelo de lenguaje la sentencia:

La polaridad de la frase “Me gusta Monica Belucci es:”

2. Miramos cual es la palabra más probable que la continúe:

$P(\text{positiva} | \text{La polaridad de la frase “Me gusta Monica Belucci es:”})$

$P(\text{negativa} | \text{La polaridad de la frase “Me gusta Monica Belucci es:”})$

# Muchas tareas pueden ponerse de esta forma

- QA (Preguntas y respuestas):

Q: ¿Quién escribió el origen de las especies?

1. Pasamos al modelo la frase:

El origen de las especies fue escrito por ...

2. Iteramos:

El origen de las especies fue escrito por Charles ...

# Decoding and Sampling

- This task of choosing a word to generate based on the model's probabilities is called **decoding**.
- The most common method for decoding in LLMs: **sampling**.
- Sampling from a model's distribution over words:
  - choose random words according to their probability assigned by the model.
- After each token we'll sample words to generate according to their probability *conditioned on our previous choices*,
  - A transformer language model will give the probability



# Random sampling doesn't work very well

- Even though random sampling mostly generate sensible, high-probable words,
- There are many odd, low- probability words in the tail of the distribution
- Each one is low- probability but added up they constitute a large portion of the distribution
- So they get picked enough to generate weird sentences

# Factors in word sampling: **quality** and **diversity**

- Emphasize **high-probability** words
  - + **quality**: more accurate, coherent, and factual,
  - **diversity**: boring, repetitive.
- Emphasize **middle-probability** words
  - + **diversity**: more creative, diverse,
  - **quality**: less factual, incoherent

# Top-k sampling:

1. Choose # of words  $k$
2. For each word in the vocabulary  $V$ , use the language model to compute the likelihood of this word given the context  $p(w_t | w_{<t})$
3. Sort the words by likelihood, keep only the top  $k$  most probable words.
4. Renormalize the scores of the  $k$  words to be a legitimate probability distribution.
5. Randomly sample a word from within these remaining  $k$  most-probable words according to its probability.

# Top-p sampling (= nucleus sampling)

- Problem with top- $k$ :  $k$  is fixed so may cover very different amounts of probability mass in different situations
- Idea: Instead, keep the top  $p$  percent of the probability mass
- Given a distribution  $P(w_t | \mathbf{w}_{<t})$ , the top- $p$  vocabulary  $V(p)$  is the smallest set of words such that

$$\sum_{w \in V(p)} P(w | \mathbf{w}_{<t}) \geq p$$

# Temperature sampling

- Reshape the distribution instead of truncating it
- Intuition from thermodynamics:
  - a system at high temperature is flexible and can explore many possible states,
  - a system at lower temperature is likely to explore a subset of lower energy (better) states.
- In **low-temperature sampling**, ( $\tau \leq 1$ ) we smoothly
  - increase the probability of the most probable words
  - decrease the probability of the rare words.
- <https://platform.openai.com/docs/api-reference/chat/create>

# Pretraining

- The big idea that underlies all the amazing performance of language models
- First **pretrain** a transformer model on enormous amounts of text
- Then **apply** it to new tasks.

# Self-supervised training algorithm

- We just train them to predict the next word!
  1. Take a corpus of text
  2. At each time step  $t$ 
    - i. ask the model to predict the next word
    - ii. train the model using gradient descent to minimize the error in this prediction

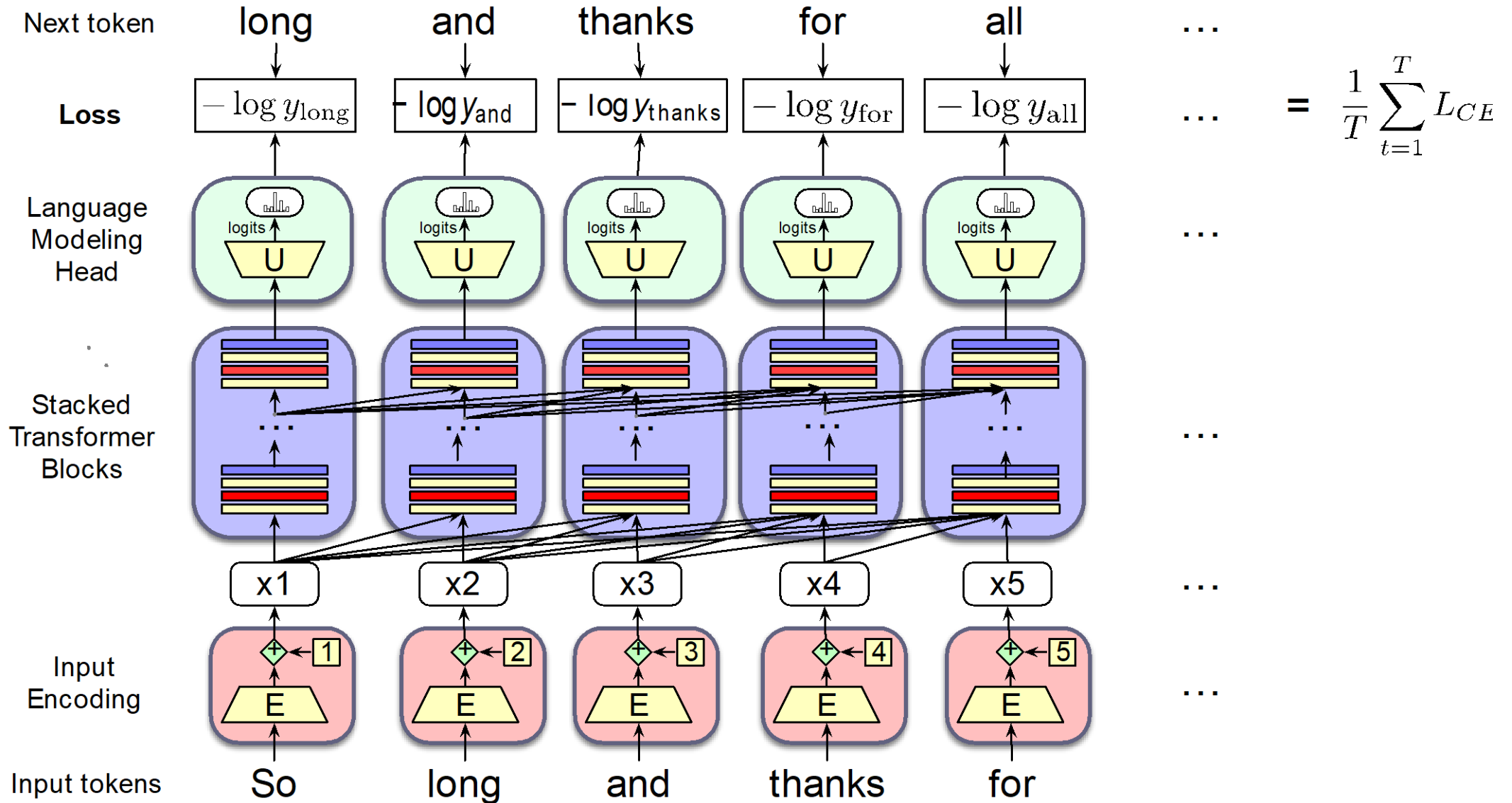
**"Self-supervised"** because it just uses the next word as the label!

# Teacher forcing

- At each token position  $t$ , model sees correct tokens  $w_{1:t}$ ,
- Computes loss ( $-\log$  probability) for the next token  $w_{t+1}$
- At next token position  $t+1$  we ignore what model predicted for  $w_{t+1}$
- Instead we take the **correct** word  $w_{t+1}$ , add it to context, move on



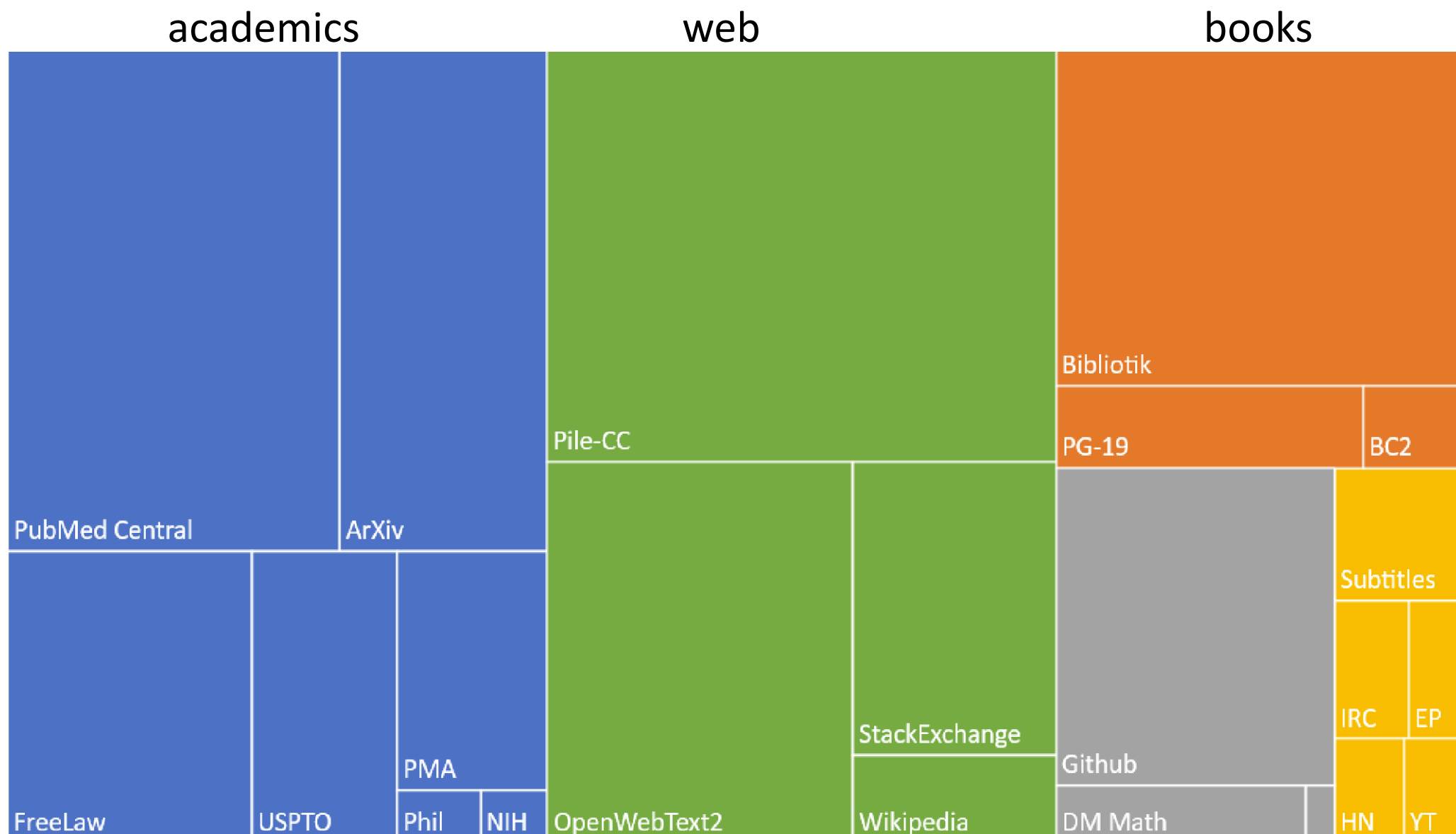
# Training a transformer language model



# LLMs are mainly trained on the web

- Common crawl, snapshots of the entire web produced by the non-profit Common Crawl with billions of pages
- Colossal Clean Crawled Corpus (C4; [Raffel et al. 2020](#)), 156 billion tokens of English, filtered
- What's in it? Mostly patent text documents, Wikipedia, and news sites

# The Pile: a pretraining corpus



# But there are problems with scraping from the web

- **Copyright:** much of the text in these datasets is copyrighted
  - Not clear if fair use allows for this use
  - This remains an open legal question
- **Data consent**
  - Website owners can indicate they don't want their site crawled
- **Privacy:**
  - Websites can contain private IP addresses and phone numbers

# Modelos fundacionales

- En el contexto de **LLMs (Large Language Models)**, un **modelo fundacional (Foundation Model)** se refiere a un modelo de inteligencia artificial extremadamente grande y complejo, entrenado con grandes cantidades de datos (habitualmente miles de millones de palabras o más), capaz de adaptarse rápidamente a diversas tareas específicas mediante ajustes mínimos o adicionales (**fine-tuning**).
- Se pueden utilizar tal y como están, pero habitualmente son el primer paso de la construcción de otros tipos de LLM. Posteriormente, se hace fine-tuning de este modelo para adaptarlo a tareas específicas:
  - Modelos instruct
  - Q&A
  - Generación de código

# Modelos instruct

Un **Modelo Instruct** es un modelo fundacional entrenado adicionalmente (**fine-tuning**) para responder efectivamente a instrucciones o solicitudes explícitas dadas por los usuarios.

Estos modelos pueden recibir instrucciones naturales como:

- *"Resúmeme el texto..."*
  - *"Explícame cómo funciona..."*
  - *"Dame ejemplos de..."*
- 
- *¿Cómo se entrenan los modelos instruct?*

# Modelos instruct

## 1. Instrucción-Tuning (Instruction-tuning)

- Se toman modelos fundacionales (por ejemplo, GPT-3, GPT-4, LLaMA, PaLM) previamente entrenados con grandes volúmenes de datos generales (internet, libros, etc.).
- Se recopilan o generan grandes conjuntos de datos con instrucciones específicas, normalmente formuladas en lenguaje natural, acompañadas de respuestas deseadas.
- Estos conjuntos de instrucciones/respuestas se usan para hacer **fine-tuning supervisado**, permitiendo que el modelo aprenda patrones de interacción instructiva explícita.

# Modelos instruct

## 2. Refinamiento con RLHF (Reinforcement Learning from Human Feedback)

Luego del ajuste inicial mediante instrucciones, los modelos más avanzados (como GPT-4) suelen recibir un refinamiento adicional utilizando **Reinforcement Learning from Human Feedback (RLHF)**:

- **Recopilación de preferencias humanas:**
  - Se generan múltiples respuestas del modelo para la misma instrucción.
  - Humanos califican y eligen cuál respuesta es mejor.
- **Entrenamiento con aprendizaje por refuerzo (RL):**
  - Usando las preferencias recopiladas, el modelo es entrenado para maximizar la satisfacción humana al generar respuestas.
  - Esto mejora considerablemente la calidad, seguridad y coherencia del modelo.



# Principales LLM

## Propietarios

1. GPT-4 y GPT-4-turbo (OpenAI y Microsoft)
2. Gemini (Google DeepMind)
3. Claude (Anthropic)
4. Cohere Command (Cohere)
5. Inflection-2.5 (Inflection AI)

## Open Source

1. Llama (Meta)
2. Mistral & Mixtral (Mistral AI)
3. DeepSeek R1 (DeepSeek)
4. Qwen (Alibaba)
5. Falcon 180B (TII - Emiratos Árabes)
6. BLOOM & BLOOMZ (BigScience)
7. OLMo (Allen Institute for AI)
8. Grok (X)

# PROMPT ENGINEERING

Ignacio Gómez  
2025

# Prompt Engineering

El **Prompt Engineering** es el arte y la ciencia de diseñar instrucciones (prompts) efectivas para guiar a los modelos de lenguaje (LLMs) hacia respuestas más precisas, relevantes y útiles.

Vamos a revisar las principales técnicas utilizadas.

# Zero-Shot Prompting

- **Definición:** Pedir al modelo una respuesta directamente a base de instrucciones, sin dar ejemplos previos.
- **Ejemplo:**
  - *Prompt:* "Explica qué es la fotosíntesis en una oración."
- **Uso:** Funciona bien con modelos avanzados como GPT-4 o Claude 3.

# Few-Shot Prompting

- **Definición:** Proporcionar algunos ejemplos antes de la pregunta para guiar al modelo.
- **Ejemplo:**
  - *Prompt:*

Ejemplo 1: "Capital de Francia? → París"

Ejemplo 2: "Capital de Japón? → Tokio"

Pregunta: "Capital de Brasil? →"

- **Uso:** Mejora resultados en tareas complejas o con formatos específicos.

# Chain-of-Thought (CoT) Prompting

- **Definición:** Pedir al modelo que "piense paso a paso" para resolver problemas lógicos o matemáticos.
- **Ejemplo:**
  - Prompt: "Si tengo 5 manzanas y regalo 2, ¿cuántas me quedan? Explica tu razonamiento."
  - Respuesta: " $5 - 2 = 3$ . Quedan 3 manzanas."
- **Uso:** Esencial para razonamiento complejo.

# Self-Consistency

- **Definición:** Pedir múltiples respuestas y seleccionar la más consistente.
- **Ejemplo:**
  - Prompt: "Genera 3 versiones de un resumen sobre inteligencia artificial y elige la más precisa."

# Tree-of-Thought (ToT)

- **Definición:** Explorar múltiples caminos de razonamiento y seleccionar el mejor (como un árbol de decisiones).

## Pregunta:

*"Una empresa quiere lanzar un nuevo producto. ¿Debe enfocarse en precio bajo, innovación o sostenibilidad?"*

## 1. Generar ramas de análisis:

1. *Rama 1:* Análisis de mercado (competencia, demanda).
2. *Rama 2:* Capacidades internas (¿pueden innovar rápido?).
3. *Rama 3:* Tendencias globales (ej.: consumidores prefieren sostenibilidad).

## 2. Proceso final: Analizando las tres ramas, diseña una estrategia comercial



# Generated Knowledge Prompting

- **Definición:** Pedir al modelo que genere información relevante antes de responder.
- **Ejemplo:**
  - Prompt: "Primero, lista 3 hechos sobre el cambio climático. Luego, escribe un resumen basado en ellos."

# Otras técnicas

- **Role Prompting:** "Eres un historiador experto. Explica las causas de la Revolución Industrial."
- **Delimitadores:** Resume el artículo siguiente en 3 puntos clave:  
      " "" "" "" ""  
      [Texto del artículo]  
      " "" "" "" ""
- **Negative Prompting:** "Explica la teoría de la relatividad sin usar términos técnicos."
- **Automatic Prompt Engineering (APE):** Usar otro LLM para generar/optimizar prompts.

# MODELOS MULTIMODALES

Ignacio Gómez  
2025

# Modelos multimodales

Un **LLM multimodal** es un modelo de lenguaje grande (LLM) que no solo procesa texto, sino también otras modalidades como imágenes, audio o incluso videos. El término "**multimodal**" significa que combina múltiples tipos o modalidades de datos para entender y generar respuestas integradas.

Mientras que un **LLM estándar** (como GPT-3.5) procesa únicamente texto, un **LLM multimodal** puede:

- Recibir entradas como imágenes, audio o video junto al texto.
- Entender la relación y contexto entre estas modalidades.
- Generar respuestas integradas (texto, imagen, audio o combinaciones de estas modalidades).

# Modelos multimodales: ¿Como funcionan?

## **Codificadores específicos (Encoders especializados):**

Cada modalidad (texto, imagen, audio) tiene su propio codificador que convierte los datos en representaciones numéricas (*embeddings*):

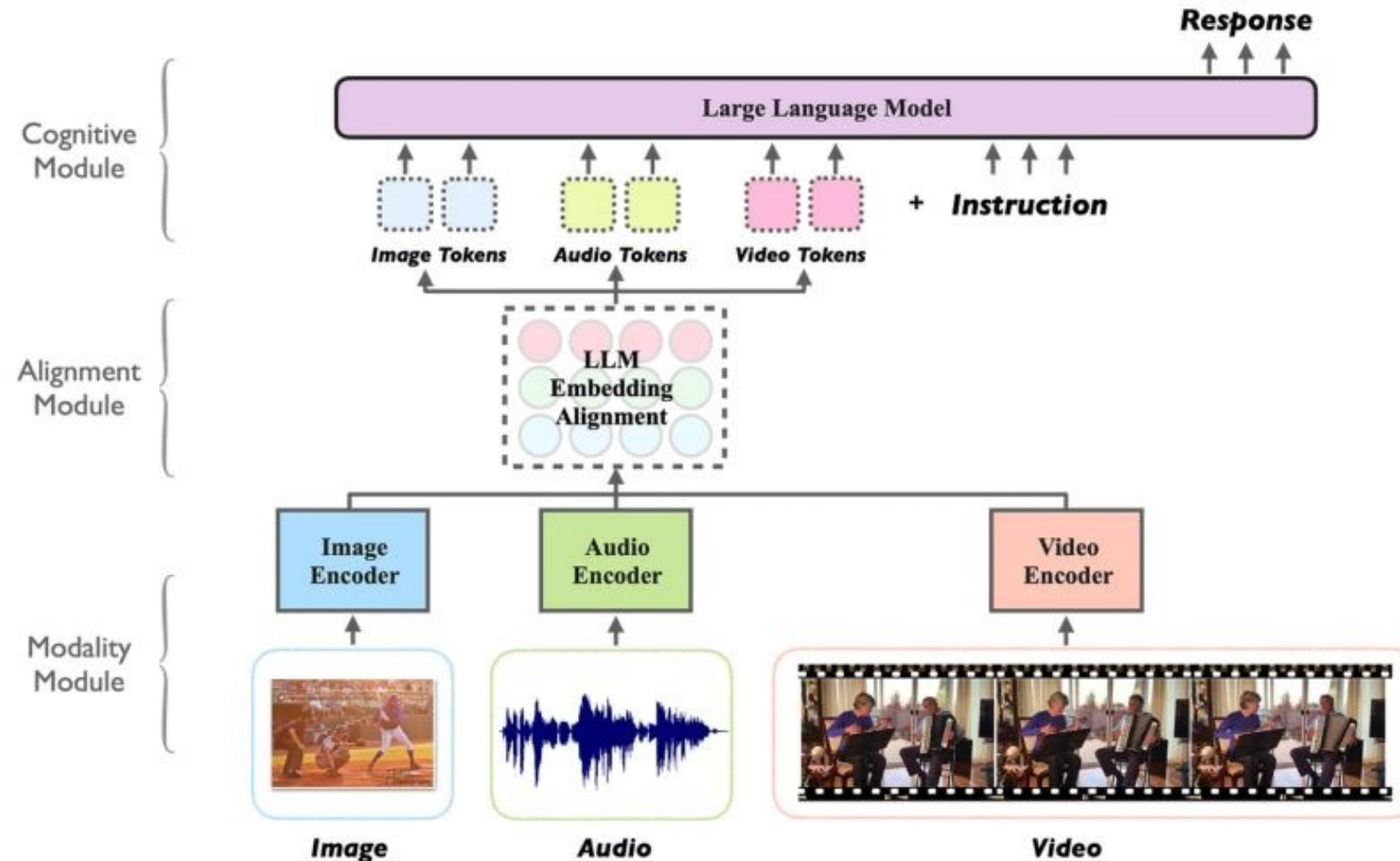
- **Texto:** codificado con modelos como Transformers (GPT, BERT).
- **Imagen:** codificado con modelos de visión como ViT (Vision Transformer), CLIP, o CNNs.
- **Audio:** codificado con modelos de audio como Whisper, wav2vec, o transformadores adaptados.

## **2. Integración y procesamiento conjunto (Fusión multimodal):**

- Las representaciones se combinan en un espacio latente compartido mediante técnicas como:
  - Concatenación de embeddings
  - Attention multimodal (atención cruzada entre modalidades)
  - Redes neuronales especializadas para fusionar vectores
- El LLM genera respuestas utilizando esta representación integrada.

# Modelos multimodales: ¿Como funcionan?

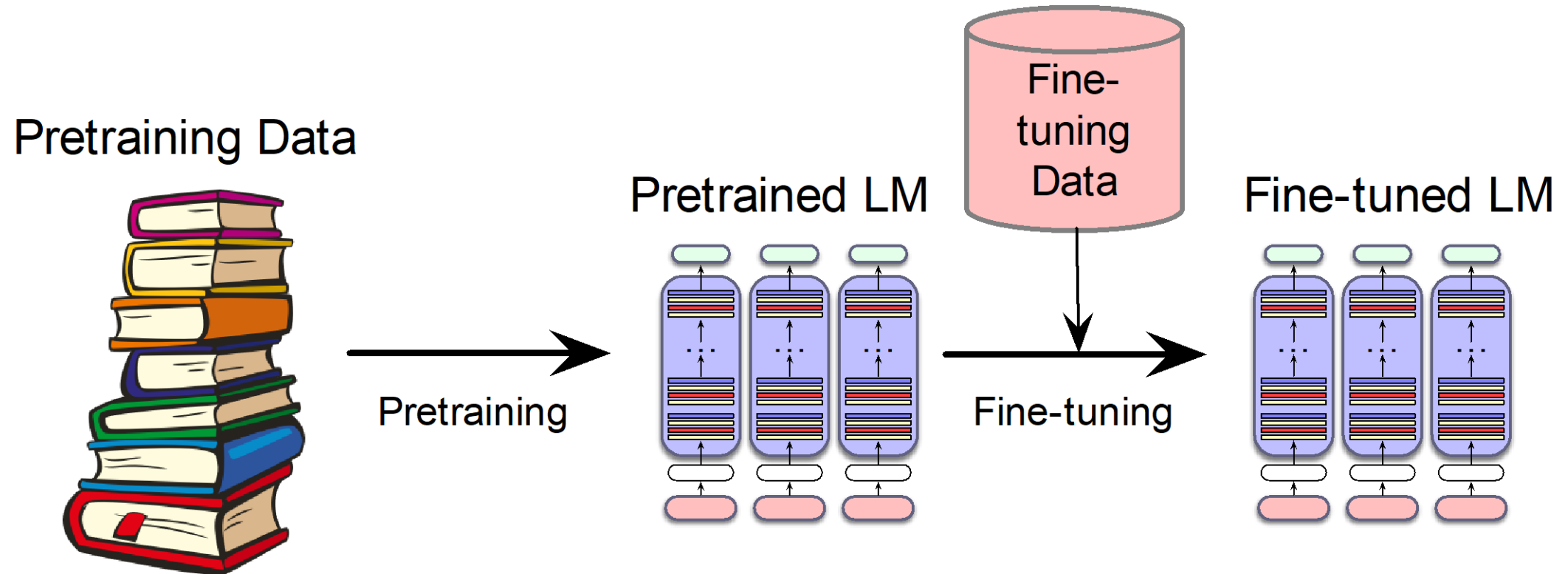
## Multimodality in Large Language Models



# FINE TUNNING

Ignacio Gómez  
2025

# Finetuning





# Perplexity

- Just as for n-gram grammars, we use perplexity to measure how well the LM predicts unseen text
- The perplexity of a model  $\theta$  on an unseen test set is the **inverse probability that  $\theta$  assigns to the test set, normalized by the test set length.**
- For a test set of  $n$  tokens  $w_{1:n}$  the perplexity is :

$$\begin{aligned}\text{Perplexity}_{\theta}(w_{1:n}) &= P_{\theta}(w_{1:n})^{-\frac{1}{n}} \\ &= \sqrt[n]{\frac{1}{P_{\theta}(w_{1:n})}}\end{aligned}$$

# Why perplexity instead of raw probability of the test set?

- Probability depends on size of test set
  - Probability gets smaller the longer the text
  - Better: a metric that is **per-word**, normalized by length
- **Perplexity** is the inverse probability of the test set, normalized by the number of words  
(The inverse comes from the original definition of perplexity from cross-entropy rate in information theory)

Probability range is  $[0,1]$ , perplexity range is  $[1,\infty]$

# Parameter-Efficient Finetuning

- Adapting to a new domain by continued pretraining (finetuning) is a problem with huge LLMs.
- Enormous numbers of parameters to train
- Each pass of batch gradient descent has to backpropagate through many many huge layers.
- Expensive in processing power, in memory, and in time.
- Instead, **parameter-efficient fine tuning** (PEFT)
  - Efficiently select a subset of parameters to update when finetuning.
  - E.g., freeze some of the parameters (don't change them),
  - And only update some a few parameters.

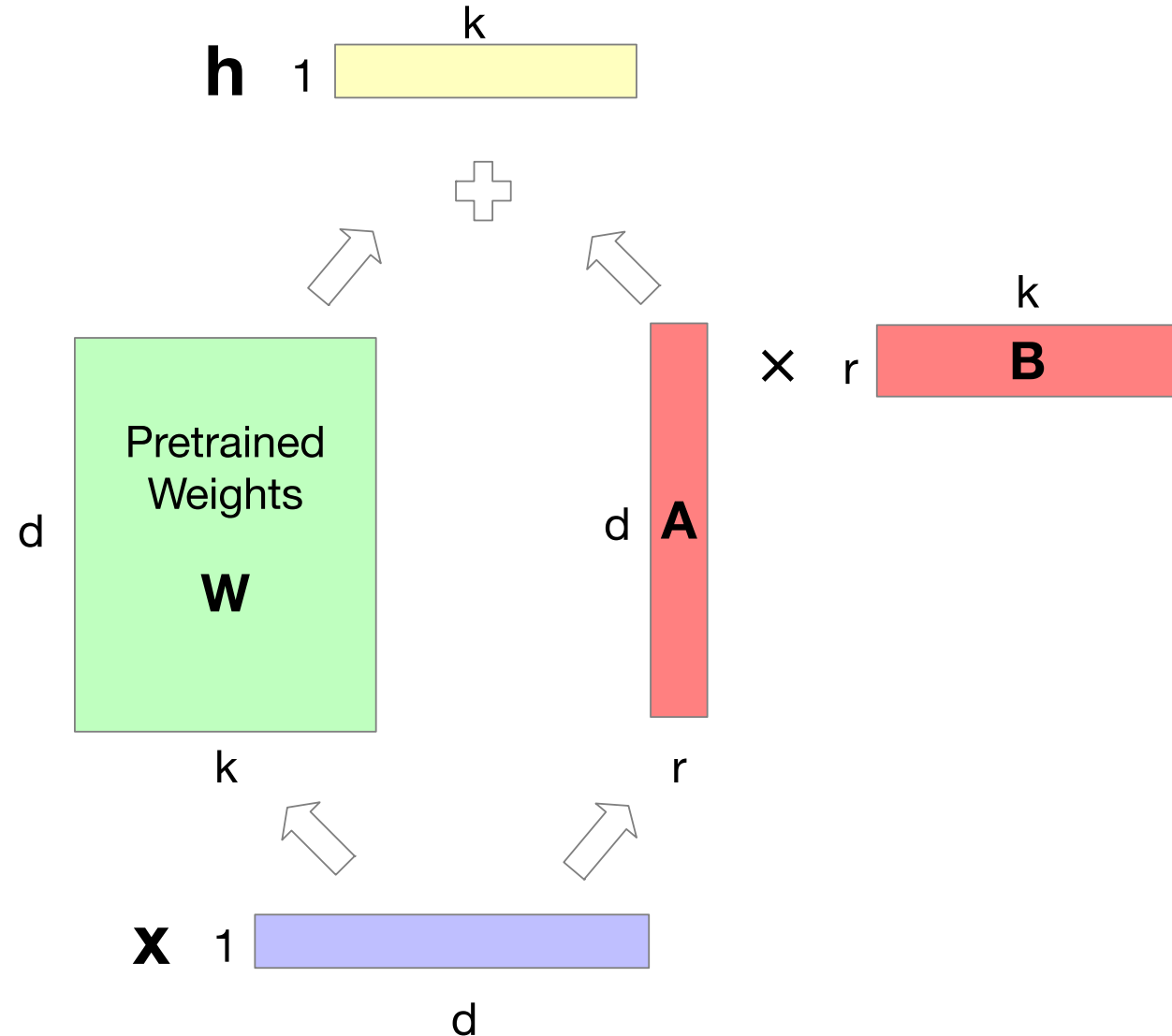
# LoRA (Low-Rank Adaptation)

- **LoRA** (Low-Rank Adaptation) es una técnica muy popular en inteligencia artificial que permite adaptar grandes modelos preentrenados (como modelos fundacionales tipo GPT, LLaMA, Stable Diffusion) para tareas específicas usando pocos recursos computacionales, memoria y datos.
- **LoRA** significa Low-Rank Adaptation, que en español sería algo así como Adaptación de Bajo Rango. Su nombre proviene del hecho de que introduce modificaciones pequeñas (de bajo rango) en los pesos del modelo original, sin alterar directamente los pesos originales.

# LoRA (Low-Rank Adaptation)

LoRA ajusta ligeramente un modelo fundacional utilizando una estrategia muy eficiente:

- **Congela** todos los pesos del modelo original (no los cambia).
- Introduce **matrices adicionales pequeñas** (de bajo rango, es decir, matrices pequeñas o de dimensiones reducidas) que se suman a las matrices de pesos originales durante la inferencia o entrenamiento.
- Estas matrices pequeñas son las únicas que se entrenan.



# Ventajas de LoRA

- **Entrenamiento Rápido:**

Al entrenar únicamente las matrices pequeñas, el ajuste es muy rápido.

- **Bajo Consumo de Memoria:**

Los modelos adaptados ocupan poco espacio de almacenamiento porque sólo se guardan las matrices pequeñas adicionales.

- **Facilidad de compartir:**

Puedes compartir adaptaciones específicas fácilmente sin necesidad de compartir todo el modelo fundacional.

- **Eficiencia en Tareas Específicas:**

Logra gran rendimiento en tareas específicas con pocos datos.

# Modelos Quantizados

- Los **modelos LLM cuantizados (Quantized Large Language Models)** son versiones comprimidas o simplificadas de modelos grandes (como GPT, LLaMA, etc.), en las que la precisión numérica de sus pesos se reduce (por ejemplo, de precisión de 32 bits a 16, 8, 4, o incluso menos bits), con el objetivo de ahorrar espacio, reducir consumo de memoria y mejorar eficiencia computacional.

Los pesos de los modelos pueden ser:

- **Precisión completa (float32)**: utiliza 32 bits por peso.
- **Media precisión (float16/bfloat16)**: utiliza 16 bits.
- **Enteros (INT8, INT4, etc.)**: usando solo 8 o 4 bits.

# ¿Cómo funciona la cuantización?

Tenemos tres opciones:

## **1. Entrenamiento original (sin cuantización):**

- Se entrena primero un modelo grande con precisión completa (float32).

## **2. Cuantización posterior al entrenamiento (Post-training quantization):**

1. Una vez entrenado, se convierten los pesos del modelo de float32 a una precisión menor (INT8, INT4).
2. Opcionalmente, se calibra usando datos representativos del dominio para minimizar pérdida de calidad.

## **3. Cuantización consciente del entrenamiento (Quantization-aware training) (*opcional pero efectivo*):**

- Durante el entrenamiento original, se simula la cuantización para que el modelo aprenda a mantener precisión aun después de reducir la precisión numérica.



## **Ventajas:**

- **Menor consumo de memoria:**  
Se reduce considerablemente el espacio necesario para almacenar modelos.
- **Mayor eficiencia computacional:**  
Procesamiento más rápido (especialmente en hardware compatible como GPUs modernas, CPUs o TPUs especializadas).
- **Menores costos operacionales:**  
Más eficiente y económico en producción.
- **Ejecutables en hardware más sencillo:**  
Permite ejecutar grandes modelos incluso en computadoras personales o dispositivos con recursos limitados.

## **Desventajas (compromisos) de la cuantización:**

- **Posible pérdida de precisión o calidad:**  
A menor número de bits, mayor riesgo de perder rendimiento original. Usualmente, INT8 es muy estable, INT4 puede requerir ajustes más finos.
- **Necesidad de calibración:**  
Es necesario un paso extra de ajuste para minimizar la pérdida en prestaciones.

# ¿Cómo funciona la cuantización?

Ejemplos populares de modelos LLM cuantizados:

- **LLaMA-2 (Meta) en INT8 o INT4:** versión muy popular para ejecutar localmente.
- **GPTQ y GGUF:** formatos comunes para distribuir modelos cuantizados.
- Modelos como **Vicuna, Alpaca, Mistral**, distribuidos en versiones INT4 o INT8.

Herramientas para cuantizar modelos LLM:

- **GPTQ, AutoGPTQ**
- **bitsandbytes** (biblioteca popular)
- **llama.cpp** (herramienta para ejecución en CPU con modelos cuantizados)
- **TensorRT, ONNX Runtime:** optimizan ejecución usando INT8 para inferencia.

# ¿Cuánta memoria necesitamos?

La memoria ocupada por un modelo se calcula principalmente por la cantidad de parámetros y la precisión numérica:

- **FP32 (32 bits)** → 4 bytes por parámetro.
- **FP16/BF16 (16 bits)** → 2 bytes por parámetro.
- **INT8 (8 bits)** → 1 byte por parámetro.
- **INT4 (4 bits)** → 0.5 bytes por parámetro.
- **Memoria del modelo = Número de parámetros × bytes por parámetro**
- Supongamos un modelo típico de 70B como LLama3 70B.
- ¿Cuántas A100 80GB necesitamos? A100 = \$20.000

# 1) Entrenamiento desde cero

Para entrenar un modelo desde cero (backpropagation), típicamente necesitas almacenar:

- Pesos del modelo
- Gradientes
- Optimizador y estados internos ( $\sim 3\times$  memoria modelo)
- **Estimación típica: 4-6 veces la memoria del modelo.**

| Precisión | Memoria (solo modelo)                               | Memoria recomendada entrenamiento |
|-----------|-----------------------------------------------------|-----------------------------------|
| FP32      | $70B \times 4 \text{ bytes} \approx 280 \text{ GB}$ | $\sim 1.1\text{--}1.7 \text{ TB}$ |
| FP16/BF16 | $70B \times 2 \text{ bytes} \approx 140 \text{ GB}$ | $\sim 560\text{--}840 \text{ GB}$ |
| INT8      | (normalmente no se usa INT8 en entrenamiento)       | -                                 |
| INT4      | (no se usa INT4 en entrenamiento)                   | -                                 |

## 2) Fine-tuning

Para hacer fine-tuning (ajuste fino) completo o parcial, también requieres almacenar:

- Pesos (pueden ser congelados parcialmente)
- Gradientes y optimizador (típicamente necesitas 3× la memoria del modelo)

Supongamos que haces **fine-tuning con LoRA** (muy eficiente):

- LoRA solo entrena una pequeña parte (~1-5%), así que prácticamente ocupas solo la memoria del modelo original + pequeña sobrecarga.

| Precisión | Fine-tuning completo (recomendado) | Fine-tuning parcial (LoRA)                           |
|-----------|------------------------------------|------------------------------------------------------|
| FP32      | ~840 GB (3× modelo)                | ~280-300 GB (modelo congelado + LoRA)                |
| FP16/BF16 | ~420 GB (3× modelo)                | ~140-160 GB (modelo congelado + LoRA)                |
| INT8      | No típico completo                 | ~70 GB aprox. (modelo INT8 congelado + LoRA FP16)    |
| INT4      | No típico completo                 | ~35-50 GB aprox. (modelo INT4 congelado + LoRA FP16) |

### 3) Inferencia

Para inferencia, solo se carga el modelo y buffers pequeños adicionales (activaciones, cache):

| Precisión | Memoria necesaria inferencia |
|-----------|------------------------------|
| FP32      | 280 GB                       |
| FP16/BF16 | 140 GB                       |
| INT8      | 70 GB                        |
| INT4      | 35 GB                        |

# Resumen

| Escenario               | FP32           | FP16/BF16      | INT8     | INT4        |
|-------------------------|----------------|----------------|----------|-------------|
| Entrenar desde cero 🚩   | 1.1–1.7 TB 🚫   | 560–840 GB 🚫   | -        | -           |
| Fine-tuning completo ⚠️ | ~840 GB 🚫      | ~420 GB 🚫      | -        | -           |
| Fine-tuning (LoRA) ✅    | ~280-300 GB ⚠️ | ~140-160 GB ⚠️ | ~70 GB ✅ | ~35-50 GB ✅ |
| Inferencia ✅            | 280 GB 🚫       | 140 GB ⚠️      | 70 GB ⚠️ | 35 GB ✅     |

- 🚫 = No viable en 1 GPU típica.
- ⚠️ = Posible pero exigente (GPUs especializadas tipo NVIDIA A100 80GB).
- ✅ = Viable y cómodo en GPU especializada de gran memoria.

# Modelos destilados

Es una analogía al proceso de destilación en química:

La destilación de conocimiento es una técnica en inteligencia artificial introducida por Geoffrey Hinton y colaboradores, en la que un \*\*modelo pequeño ("alumno" o *student*) aprende a imitar o replicar las respuestas, comportamientos o representaciones internas de un modelo grande ("profesor" o *teacher*).

- **DistilBERT:** versión reducida de BERT creada por HuggingFace, que retiene ~97% del rendimiento original de BERT Base con solo ~60% del tamaño.
- **Alpaca / Vicuna (destilados de modelos como LLaMA):** adaptaciones más pequeñas y eficientes del modelo original de Meta.
- **TinyBERT, MobileBERT:** versiones muy compactas diseñadas para uso en dispositivos móviles.



# Proceso de destilación

## 1. Entrenamiento del modelo grande (profesor)

1. Primero, entrenas un modelo muy potente, pero costoso computacionalmente (por ejemplo, GPT-4, BERT grande).

## 2. Generación de datos "soft"

- El modelo profesor genera predicciones probabilísticas (*logits*) sobre un conjunto de datos.
- Estas predicciones (probabilidades suaves o *soft labels*) contienen mucha información sobre la distribución de probabilidad que aprendió el modelo profesor.

## Entrenamiento del modelo pequeño (alumno)

- El modelo pequeño aprende imitando no sólo las etiquetas originales, sino también las predicciones probabilísticas del modelo profesor.
- Esto permite transferir conocimiento más fino y detallado, aumentando considerablemente el rendimiento del modelo pequeño.

# Bibliografía

- Daniel Jurafsky and James H. Martin. 2025. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition. Online manuscript released January 12, 2025. <https://web.stanford.edu/~jurafsky/slp3>.